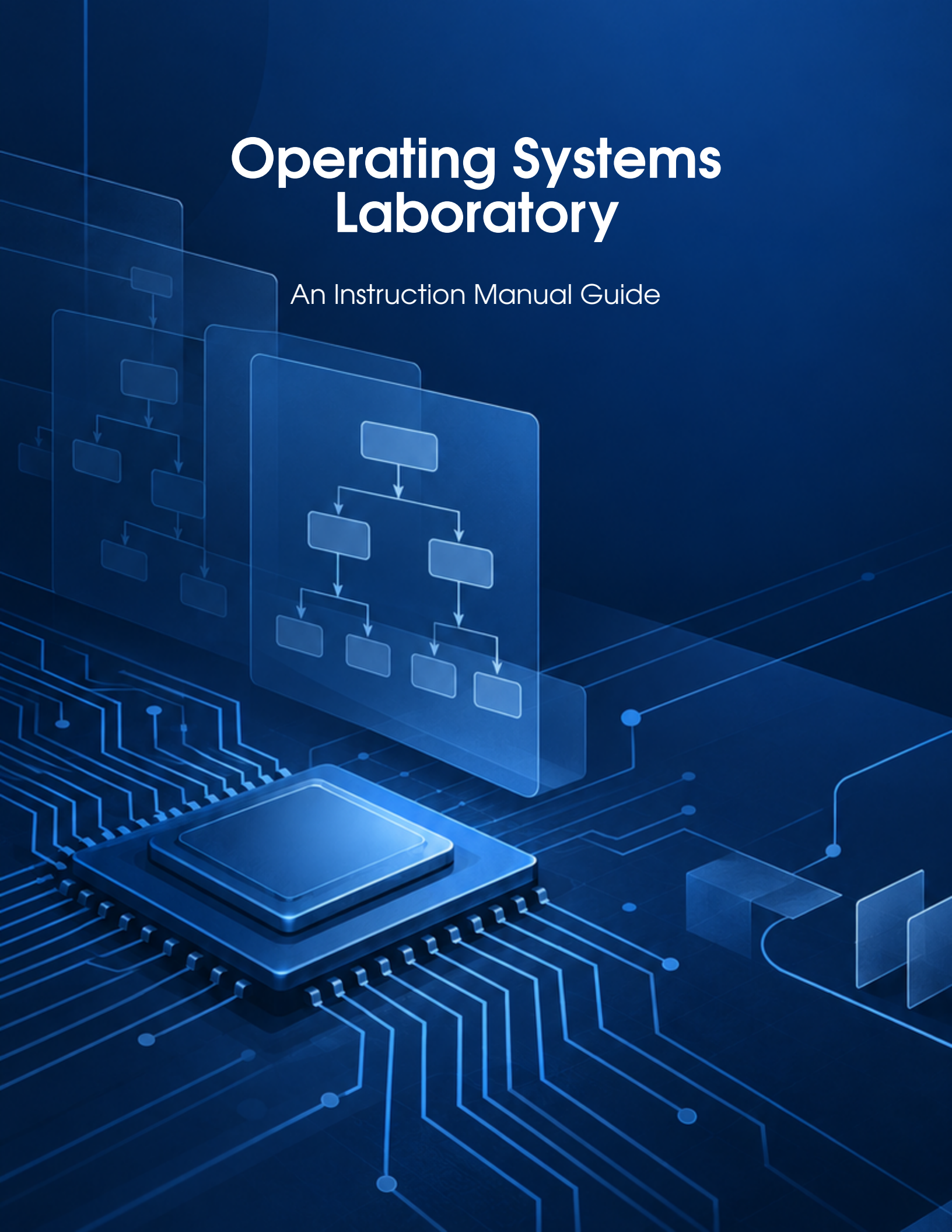
Operating Systems Laboratory

An Instruction Manual Guide



OPERATING SYSTEMS LABORATORY DEPARTMENT OF COMPUTER ENGINEERING UNIVERSITY OF MAZANDARAN

Course Instructor Dr. Afrabandpey

Teaching Assistant and Author Reza Ahmadizadeh B.Sc. Student in Computer Engineering
For questions or feedback about this manual, please contact the author at `mr.ahmadizadeh28@gmail.com`.

HTTPS://GITHUB.COM/TEOYED/OS-LAB-BOOK

This laboratory manual has been prepared for the Operating Systems Laboratory course at the University of Mazandaran. It is designed to provide students with practical instructions, laboratory exercises, and supplementary material for studying and experimenting with fundamental operating systems concepts.

The manual was prepared by Reza Ahmadizadeh in his capacity as Teaching Assistant for the course under the supervision of Dr. Afrabandpey. It is intended as a supplementary educational resource for students enrolled in the course.

This manual is maintained as an open educational resource through its accompanying Git repository. The contents may be revised and updated between editions to correct errors, improve explanations, and incorporate additional laboratory material.

First edition, 2026 University of Mazandaran

Contents

1 Virtual Machines and Linux Installation 5

1.1 What is virtualization? 5

1.2 Benefits of Virtualization 5

1.3 Main Components of Virtualization 5

1.3.1 Types of Hypervisors 6

1.4 Popular Virtualization Software 6

1.5 System Requirements for Running Virtual Machines 7

1.6 Installing Ubuntu on a Virtual Machine 8

1.7 Basic Troubleshooting 8

2 Linux File System, Command Line Interface, Users, and Groups . . . 11

2.1 Introduction to the Linux File System 11

2.1.1 What is a File System? 11

2.2 Common Linux File Systems 12

2.2.1 ext4 (Fourth Extended Filesystem) 12

2.2.2 XFS 12

2.2.3 Btrfs (B-tree Filesystem) 12

2.3 The Linux File System Hierarchy 13

2.3.1 Other Important Directories 14

2.3.2 Visual Overview 14

2.4 The Command Line Interface (CLI) 15

2.4.1 Terminal Emulators 15

2.4.2 Understanding the Prompt . . . 15
2.4.3 File and Directory Operations . . . 18
2.4.4 Combining Commands . . . 19
2.5 Users 19
2.5.1 The `sudo` Command . . . 20
2.5.2 The `/etc/passwd` and `/etc/shadow` Files . . . 20
2.5.3 User Management Commands . . . 20
2.6 Groups 21
2.6.1 Group Management Commands . . . 21
2.7 Hands-on Practices 21
2.7.1 Exercise 1: Build and manage a project directory . . . 21
2.7.2 Exercise 2: Investigate File Information and Hidden Files . . . 22
2.7.3 Exercise 3: Create and Manage Users and Groups . . . 23
2.7.4 Exercise 4: Set Up a Shared Project Directory . . . 23

3 File Permissions and Access Control . . . 25
3.1 Understanding Permissions 25
3.2 Changing Permissions 26
3.3 Special Permissions 26
3.4 Access Control Lists (ACL) 27
3.5 Hands-on Practices 28
3.5.1 Exercise 1: Change File Permissions Using Symbolic and Octal Modes . . . 28
3.5.2 Exercise 2: Change File and Group Ownership . . . 28
3.5.3 Exercise 3: Work with Special Permissions (SGID and Sticky Bit) . . . 29
3.5.4 Exercise 4: Grant Specific Access Using ACLs . . . 29

4 Working with Text Files in Linux . . . 31
4.1 Viewing Text Files 31
4.2 Editing Text Files 32
4.2.1 Using Nano . . . 32
4.2.2 Using Vim . . . 33
4.2.3 Using Gedit . . . 33
4.3 Searching in Files 34
4.3.1 Basic Usage . . . 34
4.3.2 Common Options . . . 34
4.3.3 Using Regular Expressions with Grep . . . 34
4.4 Text Processing Commands 35
4.5 Redirection and Pipes 36
4.5.1 Output Redirection . . . 36
4.5.2 Input Redirection . . . 36
4.5.3 Pipes . . . 37

4.6 Hands-on Practices **37**
4.6.1 Practice 1: Create and Edit a Text File 37
4.6.2 Practice 2: Investigate a Service Log 37
4.6.3 Practice 3: Summarize and Clean Text Data 38

Homework 1: Chapters 2, 3, and 4 39

Scenario **39**

Environment and Safety Requirements **39**

Create and Organize the Workspace **39**
Configure Team Accounts and Access 40
Investigate the Event Log and Process Text 41
Write, Review, and Publish the Handover Report 42

Explanation and Submission Checklist **42**

5 Shell Programming I 45

5.1 What is a Shell? **45**

5.2 Types of Shells **45**

5.3 Shell Interpreter Working Modes **46**

5.4 Shell Scripts **46**

5.5 Variables **47**

5.6 The `declare` Command **48**

5.7 Arrays **49**
5.7.1 Creating Arrays 49
5.7.2 Accessing Array Elements 49

5.8 Conditional Statements **50**
5.8.1 The `test` Command 50
5.8.2 Combining Conditions 50
5.8.3 The `if` Command 50

5.9 Common Test Operators **51**
5.9.1 File Tests 51
5.9.2 String Tests 51
5.9.3 Numeric Tests 51

5.10 Hands-on Practices **52**
5.10.1 Practice 1: Your First Scripts, Variables, and Arithmetic 52
5.10.2 Practice 2: Working with Arrays 52
5.10.3 Practice 3: Conditions and a File Checker Script 53

6 Shell Programming II 55

6.1 Introduction to Loops 55
6.1.1 Why Use Loops 55
6.1.2 Types of Loops in Bash 55
6.1.3 Loop Execution Flow 55

6.2 The `for` Loop 56
6.2.1 Basic Syntax and Explanation 56
6.2.2 Creating Lists for Loops 57
6.2.3 Practical Basic Examples 57
6.2.4 C-style `for` Loop 58

6.3 The `while` Loop 58
6.3.1 Basic Syntax and Explanation 58
6.3.2 Reading Files with `while` 58
6.3.3 Practical Basic Examples 59

6.4 The `until` Loop 59
6.4.1 Basic Syntax and Explanation 59
6.4.2 Comparison with `while` 60
6.4.3 Practical Basic Examples 60

6.5 Loop Control Statements 61
6.5.1 The `break` Statement 61
6.5.2 The `continue` Statement 61
6.5.3 Nested Loops 61

6.6 Advanced Loop Techniques 62
6.6.1 Processing Arrays with Loops 62
6.6.2 Command Substitution in Loops 62
6.6.3 Loop Performance Tips 63

6.7 Hands-on Practices 63
6.7.1 Practice 1: Iterating with `for` Loops 63
6.7.2 Practice 2: Repeating with `while` and `until` 63
6.7.3 Practice 3: Controlling Loops and Combining Concepts 64

7 Shell Programming III 65

7.1 What Is a Function? 65

7.2 Defining Functions 65

7.3 Calling Functions 66

7.4 Function Arguments 66

7.5 Returning Values 67

7.6 Variable Scope 67

7.7 Recursive Functions 68

7.8 Hands-on Practices 69
7.8.1 Practice 1: Defining, Calling, and Passing Arguments to Functions 69
7.8.2 Practice 2: Returning Values from Functions 70
7.8.3 Practice 3: Variable Scope and Recursive Functions 70

8 Processes in Linux and Process Scheduling 73
8.1 What is a Process and What Components Does It Have? 73
8.2 Process Control Block (PCB) 73
8.3 Process States 74
8.4 Process Management System Calls 75
8.4.1 `fork()` 75
8.4.2 `exec()` Family 76
8.4.3 `wait()` 77
8.5 Context Switching 77
8.6 Displaying Process Structure with `pstree` 78
8.6.1 Why Use `pstree`? 79
8.6.2 Basic Usage 79

9 Multithreading 81
9.1 Introduction 81
9.2 Process vs Thread 81
9.3 POSIX Threads (pthread) 82
9.4 Race Condition Problem 85
9.4.1 Solving Race Conditions with a Mutex 86

10 Inter-Process Communication (IPC) 89
10.1 Introduction 89
10.2 Pipes (Anonymous Pipes) 89
10.3 FIFOs (Named Pipes) 92
10.4 System V Message Queues 93
10.5 System V Shared Memory 96
10.6 Signals 98
10.7 Sockets 99

1. Virtual Machines and Linux Installation

1.1 What is virtualization?

Virtualization is a technology that lets you create **virtual computers** inside one physical computer. Instead of buying many physical machines, you can run several virtual machines on a single computer. This makes much better use of your hardware.

Think of it like this: one powerful physical computer can act as several separate computers at the same time.

1.2 Benefits of Virtualization

Virtualization gives many advantages, especially for learning and working with operating systems:

- **Better use of resources**: You can run many systems on one computer.
- **Easier management**: It is simpler to control and monitor everything from one place.
- **Less downtime**: If something fails, you can quickly move or restart a virtual machine.
- **Fast setup**: You can create a new virtual computer in just a few minutes.
- **Disaster recovery**: It is easier to backup and restore systems.
- **Lower cost**: You need less physical hardware and use less electricity.

1.3 Main Components of Virtualization

There are three main parts in virtualization:

1. **Physical Machine (Host)**
   This is your real computer — the actual hardware that has CPU, memory (RAM), storage, and network.

2. **Virtual Machine (VM or Guest)**
   This is a software version of a complete computer. It runs its own operating system (like Ubuntu) inside your physical computer.

3. **Hypervisor**
   This is the special software that creates and manages the virtual machines. It sits between the physical hardware and the virtual machines. It gives each VM the resources it needs (CPU, RAM, disk, etc.).

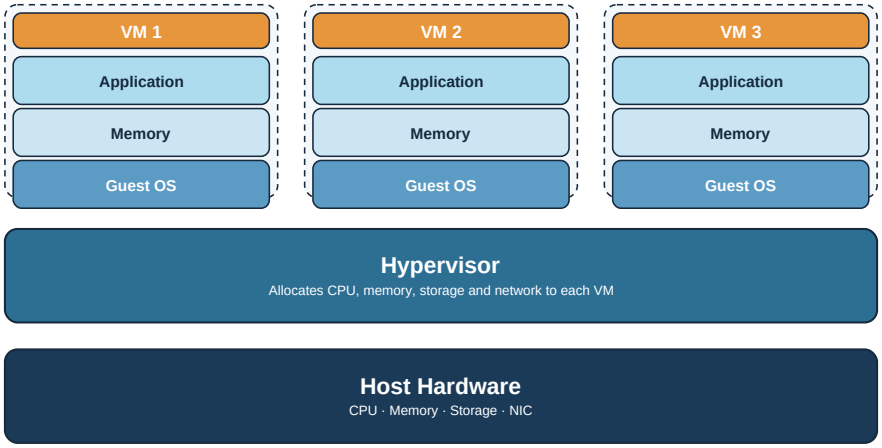


Figure 1.1: Virtualization architecture: each VM runs its own guest OS and applications on top of the hypervisor, which shares the underlying host hardware among them.

1.3.1 Types of Hypervisors

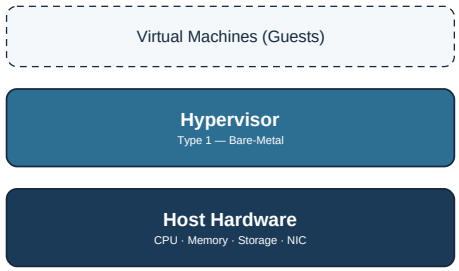
There are two main types:

- **Type 1 Hypervisors (Bare-metal)**:
  These run directly on the physical hardware. They do not need a host operating system. They are fast and powerful. Examples: VMware ESXi, Microsoft Hyper-V (in some modes), KVM.
- **Type 2 Hypervisors**:
  These run as a normal program on top of your existing operating system (Windows, Linux, or macOS). They are easier to install and good for learning. Examples: Oracle VirtualBox, VMware Workstation Player.

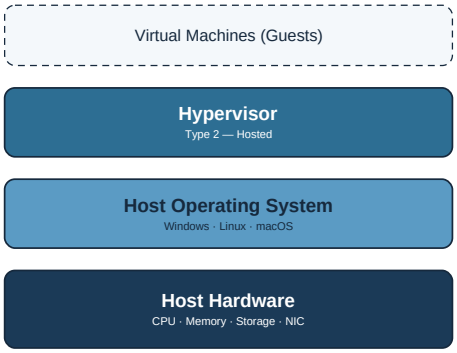
1.4 Popular Virtualization Software

For Windows users:

- Oracle VirtualBox (free and recommended for students)
- VMware Workstation Player (free for personal use)



(a) Type 1 (bare-metal): the hypervisor runs directly on the hardware.



(b) Type 2 (hosted): the hypervisor runs on top of a host operating system.

Figure 1.2: The two hypervisor types differ in what sits between the virtual machines and the physical hardware.

- Microsoft Hyper-V (available in Windows Pro)

For macOS users:

- Parallels Desktop
- VMware Fusion
- UTM (free, good for Apple Silicon)

For Linux users:

- KVM + QEMU (very efficient)
- Oracle VirtualBox

1.5 System Requirements for Running Virtual Machines

To run virtual machines smoothly, your computer should meet these requirements:

Requirement	Description
Virtualization Support	CPU must support Intel VT-x or AMD-V (usually enabled in BIOS)
Processor	64-bit processor
Memory (RAM)	At least 8 GB recommended (4 GB minimum)
Free Disk Space	At least 20 GB (more is better)

Table 1.1: System Requirements for Virtualization

Important Tip: You must enable virtualization in your computer's BIOS/UEFI settings. Without this, virtual machines will be very slow or may not work.

1.6 Installing Ubuntu on a Virtual Machine

Step-by-Step Process

1. **Download the Ubuntu ISO**
   Go to the official Ubuntu website and download the latest Ubuntu Desktop ISO file.
2. **Install Virtualization Software**
   Install VirtualBox, VMware Workstation Player, or another tool of your choice.
3. **Create a New Virtual Machine**
   - Choose "Linux" as the type.
   - Give it a name.
   - Allocate resources:
     - 2 or more CPU cores
     - 4 to 8 GB of RAM
     - 20 GB or more virtual hard disk
4. **Mount the ISO**
   Attach the downloaded Ubuntu ISO file as the bootable CD/DVD.
5. **Start the VM and Install Ubuntu**
   - Turn on the virtual machine.
   - Choose your language and keyboard.
   - Select "Install Ubuntu".
   - Use the default partition settings (for beginners).
   - Create a username and a strong password.
6. **Finish Installation**
   After installation is complete, restart the virtual machine and remove the ISO from the virtual CD drive.

1.7 Basic Troubleshooting

Here are common problems and their solutions:

1. **Installation is very slow**
   Solution: Enable virtualization (VT-x/AMD-V) in BIOS. Increase RAM and CPU cores for the VM.
2. **Black screen after installation**
   Solution: Install VirtualBox Guest Additions or VMware Tools. Enable 3D acceleration if available.

3. **No internet connection inside the VM**
   Solution: Set Network Adapter to **NAT** mode. Check host firewall if needed.

4. **Not enough disk space**
   Solution: Increase the virtual hard disk size in the VM settings.

2. Linux File System, Command Line Interface,

2.1 Introduction to the Linux File System

The Linux file system is the foundation of how your computer organizes and stores data. Think of it like a well-organized library where every book (file) has its specific place on a shelf (directory). Understanding how this system works is essential for anyone working with Linux, since almost every task in this course — from running programs to configuring services — ultimately comes down to reading, writing, or navigating files.

Key Concepts:

- **Efficient Organization:** The Linux file system arranges data in a logical, hierarchical structure. Everything has a place and a purpose, making it easy to find and manage files.
- **System Navigation:** Just like learning to navigate a new building, understanding the file system helps you move around your computer efficiently. You will learn where programs are stored, where your personal files live, and where system configurations are kept.
- **Command Line Interface (CLI):** The CLI is your direct line of communication with the file system. While graphical interfaces are user-friendly, the command line gives you powerful tools to interact with files and directories quickly and precisely.

The file system is not just about storage — it is about control. It manages **permissions** (who can access what), **tracks changes**, and **ensures data integrity**. As you progress through this chapter, you will see that the file system is the backbone of everything that happens on a Linux machine.

2.1.1 What is a File System?

A file system is like the organizational system in a filing cabinet. Just as you need a method to organize paper documents, your computer needs a system to organize digital files on disk.

Core Functions:

- **Storage Control:** The file system decides exactly how data is written to your hard drive and how it is retrieved. When you save a file, the file system determines where on the disk it goes;

when you open a file, it knows exactly where to find it.

- **Organization:** Files are grouped into directories (folders), which can contain other directories, creating a tree-like structure. This hierarchy keeps everything organized and accessible.
- **Access Management:** The file system controls permissions — who can read, write, or execute files. This security layer protects sensitive data and prevents unauthorized access. We cover this in detail in Chapter 3.
- **Metadata Management:** Beyond just storing your data, the file system tracks metadata such as file size, creation date, modification date, and ownership.

2.2 Common Linux File Systems

Linux supports several file systems, each with different strengths. The three most common are **ext4**, **XFS**, and **Btrfs**.

2.2.1 ext4 (Fourth Extended Filesystem)

A stable and widely used file system, ideal for general-purpose Linux installations.

- Default on most Linux distributions; highly reliable and well-tested
- Large file and volume support (files up to 16 TiB, volumes up to 1 EiB)
- Journaling ensures quick recovery after crashes
- Efficient performance through extents and delayed allocation
- Backward compatible with ext2/ext3

Best for: Desktops, laptops, and general-purpose servers.

2.2.2 XFS

A high-performance file system designed for handling large files efficiently.

- Excellent scalability for very large storage volumes
- Optimized for parallel I/O and high-speed data access
- Journaling for quick crash recovery
- Fast metadata operations and file system repair (via `xfs_repair`)
- Ideal for heavy workloads such as video editing or databases

Best for: High-performance systems and large data workloads.

2.2.3 Btrfs (B-tree Filesystem)

A modern, feature-rich file system built for flexibility, data protection, and advanced management.

- Built-in RAID support and disk pooling
- Snapshots and subvolumes for easy backups and rollbacks

- Data integrity checks using checksums
- Online resizing and defragmentation
- Support for transparent compression and deduplication

Best for: Systems requiring advanced storage features and data reliability.

Comparison Summary

Feature	ext4	XFS	Btrfs
Journaling	Yes	Yes	Yes
Snapshots / RAID	No	No	Yes
Performance	Balanced	Excellent for large files	Moderate
Scalability	High	Very High	High
Max File Size	Up to 16 TiB	Up to 8 EiB	Up to 16 EiB
Max Volume Size	Up to 1 EiB	Up to 8 EiB	Up to 16 EiB
Best Use Case	General-purpose systems	Large-file and high-performance workloads	Advanced storage and data integrity

Table 2.1: Comparison of Common Linux File Systems

Checking the file system of a mounted volume:

```
$ df -Th
```

The `-T` flag adds a column showing the file system type (ext4, xfs, btrfs, ...) for each mounted volume, and `-h` prints sizes in a human-readable form (GB, TB, ...).

2.3 The Linux File System Hierarchy

Linux organizes everything into a single tree structure, starting from the `root` directory (`/`). Every file, folder, and device stems from this root, forming a unified hierarchy — unlike Windows, there is no separate `C:`, `D:`, etc. drive letter; every storage device is *mounted* somewhere inside this same tree.

Root Directory (`/`): The starting point of the entire file system. All other directories branch from here.

`/home` **– User Home Directories:** Stores personal files and user-specific configuration. Each user has a separate folder under `/home`.

- Contains documents, downloads, and hidden configuration files (`.bashrc`, `.config/`)
- Each user has a private workspace, e.g. `/home/student/`
- Comparable to "My Documents" on Windows

`/etc` **– System Configuration Files:** Holds system-wide configuration and initialization files used by Linux and applications.

- Includes user and network settings (`/etc/passwd`, `/etc/network/`)
- Files are plain text and editable, but usually require root privileges to modify
- Always back up a configuration file before editing it

`/var` **– Variable Data Files:** Contains files that change frequently during system operation.

- Logs (`/var/log/`), caches (`/var/cache/`), and spools (`/var/spool/`)
- Grows over time; disk usage here should be monitored
- Useful for troubleshooting system behavior

`/tmp` **– Temporary Files:** Used for temporary data created by programs or users.

- Cleared automatically on reboot (on most distributions)
- Accessible to all users
- Not suitable for storing important data

`/dev` **– Device Files:** Represents hardware devices as special files that applications can access.

- `/dev/sda` → first hard drive, `/dev/sda1` → its first partition
- `/dev/null` → discards all data written to it
- Managed automatically by the system (via `udev`)
- Enables programs to interact with hardware through ordinary file operations

2.3.1 Other Important Directories

`/bin` **– Essential User Commands:** Core command-line utilities needed for basic system operation (e.g. `ls`, `cp`, `mv`, `mkdir`).

`/sbin` **– System Administration Commands:** System management tools for administrators (e.g. `reboot`, `shutdown`, `fdisk`).

`/usr` **– User Applications and Libraries:** Stores most user-installed software and shared resources.

- `/usr/bin/` → common programs
- `/usr/lib/` → libraries
- `/usr/share/` → documentation, icons, and other shared data

`/boot` **– Boot Files:** Contains the Linux kernel and boot loader configuration files needed to start the system.

`/opt` **– Optional Software:** Used for manually installed or third-party applications (e.g. Google Chrome, VirtualBox).

2.3.2 Visual Overview

Figure 2.1 summarizes the directories introduced above as branches of the same root tree. Note that this is a small, teaching-oriented subset — a real system also has directories such as `/proc`, `/sys`, `/lib`, `/mnt`, and `/root` (the home directory of the root user, not to be confused with `/`).

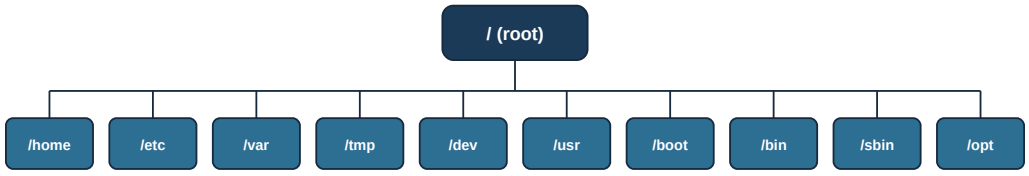


Figure 2.1: The Linux file system hierarchy: every directory branches from the single root (/).

2.4 The Command Line Interface (CLI)

The CLI allows you to interact with Linux by typing commands instead of clicking through a graphical interface. It is more **powerful**, **precise**, **fast**, and **consistent** once you get used to it, and it is the primary way you will interact with Linux throughout this course.

What is the CLI?

- A text-based interface for entering commands.
- Think of it as telling the computer exactly what to do, instead of pointing at icons.

Why Use the CLI?

- **Powerful:** Work with multiple files, chain commands, automate tasks, and access features not exposed in any GUI.
- **Precise:** Commands do exactly what you type, with no ambiguity.
- **Fast:** Once learned, typing commands is quicker than navigating menus.
- **Portable:** Works on servers, remote systems, and across Linux distributions, even without a graphical environment installed.

2.4.1 Terminal Emulators

A **terminal emulator** is the application window in which you type commands; the CLI itself is provided underneath by the **shell** (commonly `bash`). Common terminal emulators include:

- **GNOME Terminal:** default on Ubuntu/GNOME
- **Konsole:** default on KDE
- **xterm:** lightweight, classic terminal
- **Terminator:** supports split-screen layouts
- **Tilix:** tiling terminal emulator

2.4.2 Understanding the Prompt

When you open a terminal, you usually see something like:

```
student@linux:~$
```

Each part tells you important information about your session, and the command you type generally follows the same predictable pattern of `command`, `options`, and `arguments`. Figure 2.2 breaks both down visually.

1. `student` – **Username:** the account currently logged in.
2. `@` – **Separator:** separates the username from the computer name.
3. `linux` – **Hostname:** identifies the machine you are working on; useful when managing multiple computers or connecting remotely over SSH.
4. `:` – **Separator:** separates the host information from the current working directory.
5. `~` – **Current Directory:** shows the directory you are currently in (`~` is shorthand for your home directory).
6. `$` – **Prompt Symbol:** indicates the type of user:
   - `$` → regular user
   - `#` → root (administrator)

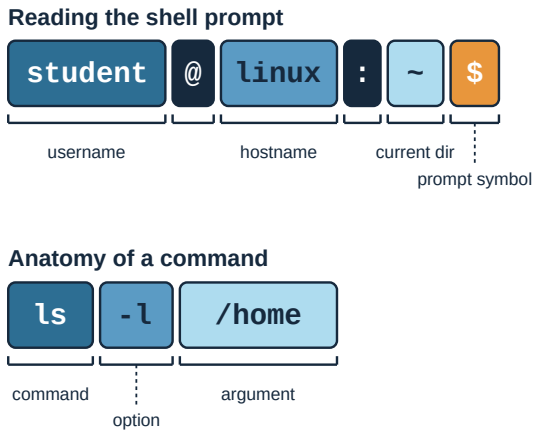


Figure 2.2: Reading the shell prompt, and the general anatomy of a command.

Basic Terminal Usage

Steps to run a command:

1. Type a command at the prompt.
2. Press `Enter` to execute it.
3. View the output.
4. Receive a new prompt for the next command.

Command Structure:

```
command [options] [arguments]
```

- `command`: the action you want to perform
- `options` (or *flags*): modify how the command behaves (optional)
- `arguments`: the files, directories, or targets the command acts on (optional)

Example:

```
$ ls -l /home
```

- `ls` → list directory contents
- `-l` → long format (detailed info)
- `/home` → directory to list

Getting Help: Almost every command documents itself. Before guessing what a flag does, check:

```
$ man ls          # full manual page (press q to quit)
$ ls --help       # short usage summary
```

Navigation Commands

`pwd` **– Print Working Directory:** shows your current directory.

```
$ pwd
/home/student
```

`ls` **– List Directory Contents:** shows files and directories in the current location.

```
$ ls
Documents  Downloads  Pictures
```

Common options:

- `ls -l` → detailed (long) list
- `ls -a` → include hidden files (names starting with `.`)
- `ls -lh` → long list with human-readable sizes

Tip: Options can be combined: `ls -lah` → long, all files, human-readable sizes.

`cd` **– Change Directory:** moves between directories.

```
$ cd Documents       # go into Documents
$ cd ..              # go up one level (parent directory)
$ cd /               # go to the root directory
$ cd ~               # go to the home directory
$ cd -               # go to the previous directory
```

Absolute path: starts from the root (/), e.g. `/home/student/Documents`.
Relative path: starts from the current directory, e.g. `Documents/report.txt`.

2.4.3 File and Directory Operations

`mkdir` **– Create Directories:**

```
$ mkdir project              # create a single directory
$ mkdir -p project/src/lib   # create nested directories in one step
```

Tip: A directory name containing spaces (e.g. `lab practice`) must be quoted or escaped, e.g. `mkdir "lab practice"` or `mkdir lab\ practice` – otherwise the shell treats it as two separate arguments.

`rmdir` **– Remove Empty Directories:**

```
$ rmdir empty_folder
```

`rmdir` only works on empty directories; to remove a directory and everything inside it, use `rm -r` (below).

`touch` **– Create a File:** creates an empty file, or updates its timestamp if it already exists.

```
$ touch newfile.txt
$ touch file1.txt file2.txt file3.txt
```

`cp` **– Copy Files/Directories:**

```
$ cp file.txt /home/student/Documents/       # copy a file
$ cp -r folder1/ folder2/                    # copy a folder recursively
$ cp -p file.txt backup.txt                  # preserve permissions and
    timestamps
$ cp -i file.txt existing.txt                # ask before overwriting
```

Tip: Use the wildcard `*` to match multiple files, e.g. `cp *.txt /backup/` copies every `.txt` file in the current directory.

`mv` **– Move/Rename Files:**

```
$ mv oldname.txt newname.txt                 # rename a file
$ mv file.txt /home/student/Documents/       # move a file
$ mv folder1/ /new/location/                 # move a folder
$ mv -i file.txt existing.txt                # ask before overwriting
```

Note: Unlike `cp`, `mv` does not leave a copy behind — the original is removed from the source location.

`rm` **– Remove Files/Directories (permanent!):**

```
$ rm file.txt                # delete a file
$ rm -r folder/              # delete a directory recursively
$ rm -f file.txt             # force delete, no confirmation
$ rm -rf folder/             # force recursive delete
```

⚠ DANGER ZONE:

```
$ rm -rf /
```

Never run this. It recursively force-deletes every file the current user can reach, starting from the root directory. Modern versions of `rm` refuse to run this exact command on `/` without an extra flag, but close variants (e.g. with a typo'd path, or run as root) can still destroy a system with no way to undo it. There is no "Recycle Bin" on the command line: `rm` deletes immediately and permanently.

2.4.4 Combining Commands

The shell lets you combine simple commands into more powerful ones:

- `cmd1 ; cmd2` – run `cmd1`, then `cmd2`, regardless of whether `cmd1` succeeded.
- `cmd1 && cmd2` – run `cmd2` only if `cmd1` succeeded.
- `cmd1 | cmd2` – a **pipe**: send the output of `cmd1` directly into `cmd2` as its input.
- `cmd > file` – redirect a command's output into `file`, overwriting it.
- `cmd » file` – redirect a command's output into `file`, appending to it.

```
$ mkdir logs && cd logs        # only enter logs/ if it was created successfully
$ ls -l | grep ".txt"          # list files, then filter for ones containing ".
    txt"
$ echo "done" >> status.log    # append a line to a log file
```

Lab Tips:

- Always check the prompt before running a command. It shows who you are and where you are, which helps prevent mistakes, especially with `rm`, `mv`, or other destructive commands.
- When connected to a remote machine via SSH, the hostname part of the prompt is especially helpful to avoid running a command on the wrong machine.
- Use `Tab` completion to save typing and avoid typos in file names.
- Use the `Up` arrow (or `history`) to reuse previous commands instead of retyping them.

2.5 Users

Users are the entities that log into the system or own running processes — in other words, everything that happens on Linux happens *as* some user. Each user is identified by a unique numerical **User ID (UID)**.

1. **Root User:** The root user has the highest level of access and can read or modify any file, use any command, and control any system resource. The root user's UID is always **0**.
2. **Regular Users:** Regular users have limited access to files and resources. They cannot perform sensitive operations, such as installing or modifying system-level software, without explicit permission. On most modern distributions, regular user UIDs start at **1000**.

3. **System Users:** System users are created to run services and background processes, such as database servers. They usually cannot log in interactively and have access only to the resources their specific service needs. System user UIDs are typically in the range **1–999**.

2.5.1 The `sudo` Command

When Linux is installed, the first user is created as a regular user, and direct login to the root account is usually disabled entirely. Users allowed to run `sudo` are listed in the **sudoers** file (`/etc/sudoers`); the first user created during installation is normally added there automatically.

Using `sudo`, an authorized user can temporarily act as root (`UID = 0`) and gain full system access for a single command:

```
$ sudo <intended command>
```

Security Note

When you run `sudo`, the system asks for **your own password**, not the root password. If your account is not listed in `/etc/sudoers`, the command fails with:

```
user is not in the sudoers file.  This incident will be reported.
```

2.5.2 The `/etc/passwd` and `/etc/shadow` Files

`/etc/passwd`**:** contains basic account information such as username, UID, home directory, and default shell. It is readable by all users.

`/etc/shadow`**:** contains encrypted passwords, along with password expiration and account restriction data. It can only be read by the root user or specific privileged system programs.

2.5.3 User Management Commands

1. `useradd`**:** adds a new user.

```
$ sudo useradd username
$ sudo useradd -m -s /bin/bash username
```

`-m`: creates a home directory for the new user.
`-s`: sets the user's default shell.

2. `passwd`**:** sets or changes a user's password.

```
$ sudo passwd username
```

3. `userdel`**:** deletes a user.

```
$ sudo userdel username
$ sudo userdel -r username   # also delete the user's home directory
```

2.6 Groups

Groups are collections of users who share access to system resources. Each group is identified by a unique numerical **Group ID (GID)**.

1. **Primary Group:** every user has one primary group, recorded in `/etc/passwd`. On many distributions, creating a new user automatically creates a new group with the same name and makes the user its only member — the **User Private Group** policy, which simplifies access management.
2. **Secondary Groups:** a user can additionally belong to any number of secondary groups, which grant extra permissions for shared resources. For example, everyone on a project team can be added to a shared `project` group so that they can all read, modify, or write the project's files.

2.6.1 Group Management Commands

1. `groupadd`**:** adds a new group.

```
$ sudo groupadd groupname
```

2. `groupdel`**:** deletes a group.

```
$ sudo groupdel groupname
```

3. `gpasswd`**:** adds or removes a user from a group.

```
$ sudo gpasswd -a username groupname   # add
$ sudo gpasswd -d username groupname   # remove
```

4. `getent`**:** looks up the members of a group.

```
$ getent group groupname
```

Tip: `groups username` and `id username` are quick ways to see which groups a user currently belongs to.

2.7 Hands-on Practices

Bellow you see a list of tasks. Complete the tasks using the Linux terminal and record the commands you use. Where requested, explain the results of your commands.

2.7.1 Exercise 1: Build and manage a project directory

Create the following directory structure inside your home directory:

```
os-lab/
|-- documents/
| |-- report.txt
|  -- notes.txt
```

```
|-- scripts/
 -- backup/
```

Complete the following tasks:

1. Create the directory structure, including all required subdirectories.
2. Create report.txt, notes.txt, and file named todo.txt inside scripts/.
3. Rename notes.txt to meeting-notes.txt.
4. Copy report.txt into backup/.
5. Move todo.txt from scripts/ to documents/.
6. Create an empty directory named empty-dir/ and remove it.
7. Display the final directory tree or an equivalent recursive listing to verify your work.

Now, answer the following questions:

- What is the difference between copying and moving a file?
- Why does `rmdir` fail when a directory is not empty?
- What happens if you copy a file to a destination where a file with the same name already exists?

2.7.2 Exercise 2: Investigate File Information and Hidden Files

Perform the following tasks inside your `os-lab/` directory:

1. Create three text files named `data1.txt`, `data2.txt`, and `data3.txt`, and one file named `README`.
2. Add different lines of text to the three `.txt` files. Ensure that the files contain different numbers of lines.
3. Create a hidden file named `.lab-config`.
4. List the directory contents normally and then list them again, including hidden files.
5. Display a detailed listing containing file permissions, ownership, file sizes, and modification times.
6. Copy all `.txt` files into the `backup/` directory using a wildcard.
7. Display the number of lines in each `.txt` file and identify which file contains the most lines.

Now, answer the following questions:

- Why is `.lab-config` not displayed by a normal directory listing?
- What information can you obtain from a long-format directory listing?
- How does the shell expand `*.txt` before executing a command?

2.7.3 Exercise 3: Create and Manage Users and Groups

Create two temporary users named `Amir` and `Reza`, each with a home directory and Bash as the login shell. Then complete the following tasks:

1. Create a group named `osproject`.
2. Add both users to `osproject` as supplementary-group members.
3. Verify that both users exist. Inspect their UIDs, primary GIDs, home directories, and supplementary groups.
4. Set passwords for the accounts if required by your instructor.
5. Remove `Reza` from `osproject` and verify the change.
6. Restore `Reza`'s membership in `osproject` if necessary for Exercise 4.
7. At the end of the lab, remove the temporary users and group after verifying that you are deleting only the accounts and group created for this exercise.

Now, answer the following questions:

- What is the difference between `useradd` and `groupadd`?
- Why might a user need to log out and log back in before newly assigned supplementary-group membership is reflected in an existing session?
- What is the difference between deleting a user and deleting a user together with their home directory?

2.7.4 Exercise 4: Set Up a Shared Project Directory

Assume that `Amir` and `Reza` are members of the supplementary group `osproject`.

1. Create a shared directory named `/srv/osproject`.
2. Set its group ownership to `osproject`.
3. Configure its permissions so that the owner and group can access the directory, while other users have no access.
4. Configure the directory so that new files and subdirectories inherit the `osproject` group.
5. As `Amir`, create a file named `shared.txt` in the directory and add some text to it.
6. As `Reza`, verify that you can read and modify `shared.txt`.
7. As `Reza`, create a second file in the shared directory.
8. Inspect the ownership and permissions of the directory and both files. Explain any differences you observe.
9. Attempt to access the directory as an unrelated ordinary user, if one is available in the lab, and verify that access is denied.
10. Explain how you would modify the configuration if project members should be able to read and modify project files while unrelated users should retain read-only access.

Now, answer the following questions:

- What is the difference between file ownership and group ownership?
- Why might group write permission on a directory be necessary even if users can write to an existing file?
- Does setting the setgid bit on a directory automatically make every new file group-writable?

3. File Permissions and Access Control

In Linux, every file and directory carries a set of permissions that decide who can read it, change it, or run it. This chapter builds on the users and groups introduced in Chapter 2: permissions are what actually put those user and group identities to work, controlling access at the level of individual files and directories.

3.1 Understanding Permissions

In Linux, every file and directory has three types of permissions:

1. **Read** (`r`) – permission to view the contents of a file, or list the contents of a directory.
2. **Write** (`w`) – permission to modify a file, or create/delete entries inside a directory.
3. **Execute** (`x`) – permission to run a file as a program, or enter (`cd` into) a directory.

These permissions are tracked separately for the file's **owner**, its **group**, and **other** users, and are displayed together as a 10-character string such as `-rwxr-xr-`. Figure 3.1 breaks this string down.

Example: `-rwxrw-r-`

- Owner: can read, write, and execute the file.
- Group: can read and write the file, but not execute it.
- Other users: can only read the file.

The leading character shows the entry type: `-` for a regular file, `d` for a directory (other values exist for links and devices, which are outside the scope of this lab).

Displaying Permissions:

```
$ ls -l file.txt
```

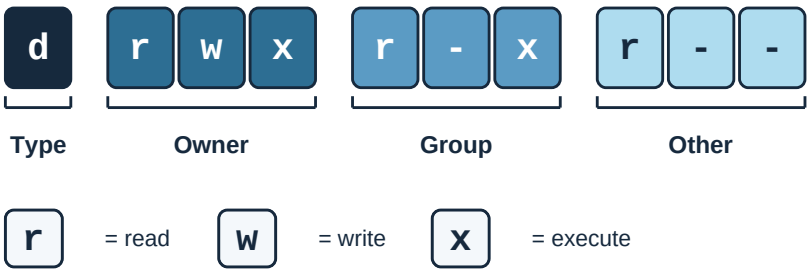


Figure 3.1: Anatomy of a permission string: file type, followed by owner, group, and other permissions.

3.2 Changing Permissions

1. **Symbolic mode:**
   + adds a permission, - removes a permission, = sets the permission exactly (removing any others).

```
$ chmod u+x file.sh    # owner gets execute permission
$ chmod g-w file.txt   # remove write permission from the group
$ chmod o=r file.txt   # others get read-only, nothing else
```

2. **Octal (numeric) mode:** each permission has a numeric value – `r = 4`, `w = 2`, `x = 1` – and the three are added together for each of owner/group/other.

```
$ chmod 754 script.sh
```

   This means:
   Owner: $7 = 4+2+1 \rightarrow$ `rwx`
   Group: $5 = 4+0+1 \rightarrow$ `r-x`
   Other: $4 = 4+0+0 \rightarrow$ `r-`

3. **Changing ownership:**

```
$ sudo chown username file.txt              # change the file's owner
$ sudo chown username:groupname file.txt    # change owner and group
    together
$ sudo chgrp groupname file.txt             # change only the group
```

3.3 Special Permissions

Beyond the standard `rwx` bits, Linux defines **three special permissions** for specific capabilities:

1. **SUID (Set User ID):** when an executable file with the SUID bit set is run, it executes with the

privileges of the file's *owner*, not the user who launched it.
Symbolic: `-rwsr-xr-x` – the `s` replaces the owner's `x`.
Numeric: `chmod 4755 /path/to/exe` – the leading `4` sets SUID.

2. **SGID (Set Group ID):**

   - On a file: it executes with the privileges of the file's *group*.
   - On a directory: new files created inside automatically inherit that directory's group, which is very useful for shared team folders.

   Symbolic: `drwxrws--` – the `s` replaces the group's `x`.
   Numeric: `chmod 2750 /opt/project` – the leading `2` sets SGID.

3. **Sticky Bit:** typically used on shared directories (e.g. `/tmp`); it restricts deletion so that only a file's owner (or root) can remove it, even if others have write access to the directory.
   Symbolic: `drwxrwxrwt` – the `t` replaces the others' `x`.
   Numeric: `chmod 1777 /tmp` – the leading `1` sets the Sticky Bit.

Note: These leading digits can be combined, e.g. `chmod 6755` sets both SUID and SGID.

3.4 Access Control Lists (ACL)

Standard Unix permissions only distinguish owner/group/other, which is not always flexible enough. **ACLs** let you assign different permissions to any number of individual users or groups on the same file.

1. **Viewing an ACL:**

```
$ getfacl file.txt
```

2. **Adding ACL permissions:**

```
$ setfacl -m u:alice:rwx file1.txt   # grant alice rwx on file1.txt
$ setfacl -m g:staff:r-- file1.txt   # grant group "staff" read-only
```

3. **Setting a default ACL for a directory** (applied automatically to new files created inside it):

```
$ setfacl -dm u:username:rwx dir/
```

4. **Removing ACL entries:**

   - Remove one user's entry: `setfacl -x u:alice file.txt`
   - Remove one group's entry: `setfacl -x g:staff file.txt`
   - Remove all ACL entries (revert to standard permissions): `setfacl -b file.txt`

Important Notes

Any command that modifies system files or user/group settings usually **requires** `sudo` – this includes `groupadd`, `useradd`, `setfacl`, `chown`, and `chmod` on files you do not own. Read-only commands such as `getfacl`, `ls`, `groups`, and `id` do **not** require `sudo`, unless the target file's permissions already block your user from reading it.

3.5 Hands-on Practices

Below you see a list of tasks. Complete the tasks using the Linux terminal and record the commands you use. Where requested, explain the results of your commands.

3.5.1 Exercise 1: Change File Permissions Using Symbolic and Octal Modes

Perform the following tasks inside your `os-lab/` directory (create it in your home directory if it does not already exist):

1. Create three empty files named `script.sh`, `notes.txt`, and `secret.txt`.
2. Display the current permissions of all three files using `ls -l`.
3. Using **symbolic mode**, add execute (`x`) permission for the owner on `script.sh`.
4. Using **symbolic mode**, remove write (`w`) permission for the group on `notes.txt` and set others' permission to read-only (`o=r`).
5. Using **octal mode**, set the permissions of `secret.txt` to 600 so that only the owner can read and write it.
6. Set `script.sh` to 754 using octal mode and verify the permissions of all three files with `ls -l`.

Now, answer the following questions:

- What do the permissions `r`, `w`, and `x` allow on a regular file versus a directory?
- What is the difference between the `+`, `-`, and `=` operators in symbolic `chmod` mode?
- What permissions does `chmod 754` grant to the owner, group, and other users?

3.5.2 Exercise 2: Change File and Group Ownership

Assume that the users `Amir` and `Reza` and the group `osproject` from Chapter 2 exist on your system (create them first if needed). Then complete the following tasks inside `os-lab/`:

1. Create a file named `project-plan.txt` and inspect its current owner and group with `ls -l`.
2. Change the owner of `project-plan.txt` to `Amir` using `chown`.
3. Change only the group of `project-plan.txt` to `osproject` using `chgrp`.
4. Change both the owner and group of `notes.txt` to `Reza` and `osproject` in a single `chown` command.
5. Run `ls -l` to verify the updated owner and group for both files.

Now, answer the following questions:

- What is the difference between `chown` and `chgrp`?
- How do you change both the owner and the group of a file in a single command?
- Why do `chown` and `chgrp` usually require `sudo`?

3.5.3 Exercise 3: Work with Special Permissions (SGID and Sticky Bit)

Assume that `Amir` and `Reza` are both members of the `osproject` group. Complete the following tasks:

1. Inspect the permissions of `/tmp` using `ls -ld /tmp` and observe the Sticky Bit (`t`) at the end of the permission string.
2. Create a shared directory named `/srv/shared-lab` and change its group ownership to `osproject`.
3. Set the permissions of `/srv/shared-lab` to `2770` so that the owner and group have full access and the **SGID** bit is enabled.
4. As `Amir`, create a file named `amir.txt` inside `/srv/shared-lab` and check with `ls -l` that it automatically inherited the `osproject` group.
5. Enable the **Sticky Bit** on `/srv/shared-lab` (for example, using `chmod 3770 /srv/shared-lab`) and check the directory permissions with `ls -ld`.
6. As `Reza`, try to delete `amir.txt` and verify that the Sticky Bit prevents you from deleting a file owned by another user.

Now, answer the following questions:

- What happens when a user runs an executable file that has the SUID bit set?
- Why is setting the SGID bit useful on a shared team directory?
- How does the Sticky Bit protect files inside a shared directory such as `/tmp`?

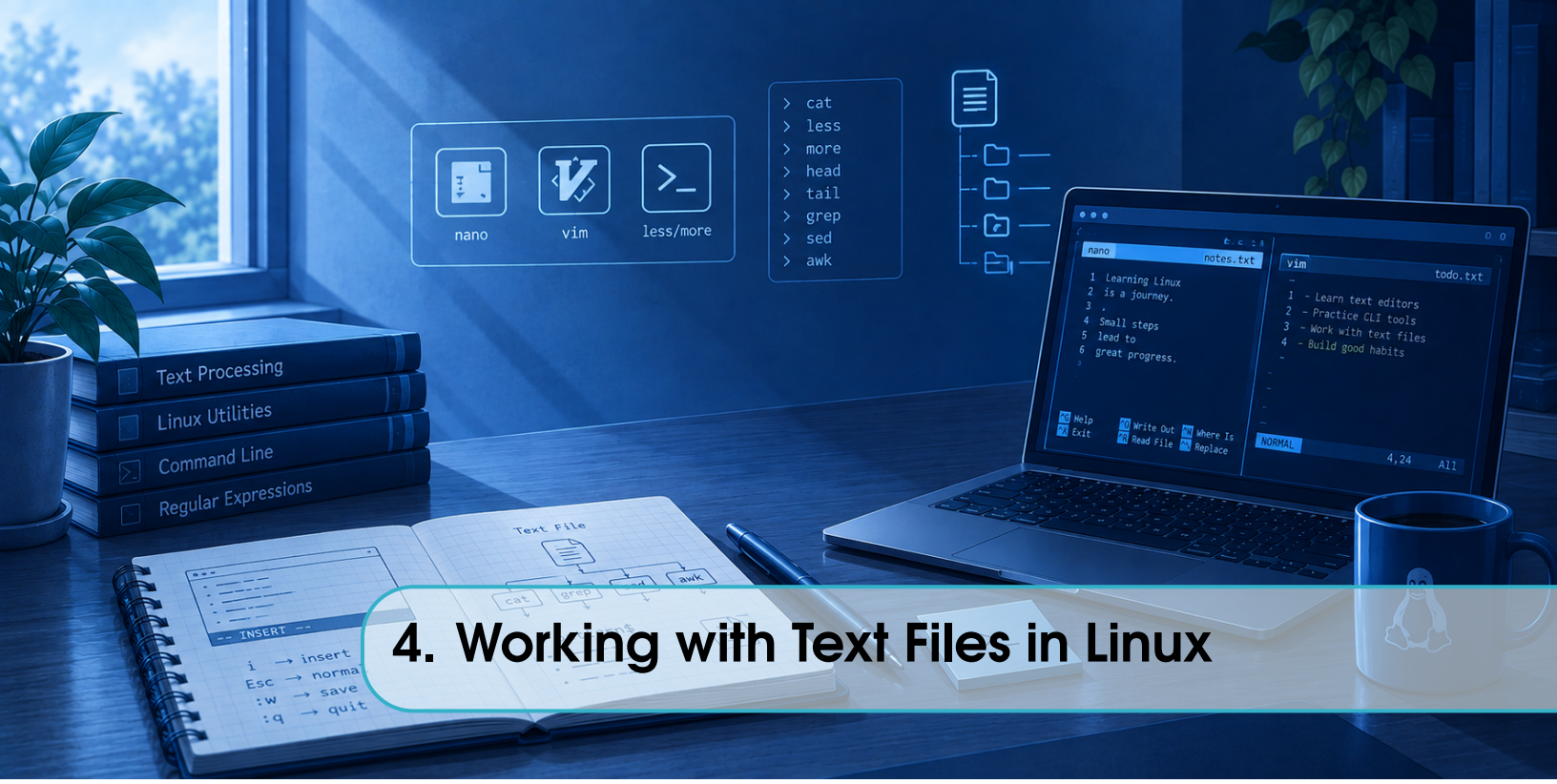
3.5.4 Exercise 4: Grant Specific Access Using ACLs

Perform the following tasks inside your `os-lab/` directory:

1. Create a file named `report.txt` and view its initial ACL entries using `getfacl`.
2. Use `setfacl` to grant the user `Amir` read, write, and execute (`rwx`) permissions on `report.txt`.
3. Use `setfacl` to grant the group `osproject` read-only (`r-`) permission on `report.txt`.
4. Run `getfacl report.txt` to verify that both ACL entries were added.
5. Set a **default ACL** on the `os-lab/` directory so that `Amir` automatically gets `rwx` permissions on any new file created inside it, then create `newfile.txt` and check its ACL with `getfacl`.
6. Remove `Amir`'s ACL entry from `report.txt` using `setfacl -x`, and then remove all ACL entries from `report.txt` using `setfacl -b`.

Now, answer the following questions:

- Why are ACLs useful when standard owner, group, and other permissions are not enough?
- What is the difference between `setfacl -m` and `setfacl -dm`?
- What is the difference between `setfacl -x` and `setfacl -b`?



4. Working with Text Files in Linux

In Linux, text files form the foundation of nearly every system operation. Configuration files, shell scripts, system logs, and most forms of data are stored as plain text. Therefore, the ability to view, edit, search, and process text files efficiently is an essential skill for any Linux user or system administrator.

This chapter introduces the common command-line tools used to interact with text files: how to read file contents, modify text data, search using patterns, and process text using filters and redirection.

4.1 Viewing Text Files

To inspect file contents, Linux provides multiple commands suited for different purposes:

- `cat` – displays the entire content of a file:

```
$ cat <file>
```

- `less` / `more` – display large files one screen at a time, letting you scroll through the file (press `q` to quit):

```
$ less <file>
$ more <file>
```

  `less` is the more modern of the two – it lets you scroll both forward and backward and doesn't need to load the whole file up front, so prefer it over `more` when both are available.

- `head` – shows the first 10 lines of a file:

```
$ head -n 10 <file>
```

- `tail` – shows the last 10 lines of a file:

```
$ tail -n 10 <file>
```

- `tail -f` – continuously monitors a file for new lines as they are written, without exiting; ideal for watching a log file in real time:

```
$ tail -f <logfile>
```

4.2 Editing Text Files

Linux provides several text editors that can be used directly from the terminal or through a graphical interface. Each editor offers a different balance of simplicity and power.

Some important text editors are:

- `nano`: a simple, beginner-friendly terminal-based editor.
- `vim`: a powerful modal editor for advanced users.
- `gedit`: a graphical text editor suitable for desktop environments.

Note: Editing system files may require administrative privileges; use `sudo` if necessary.

Opening an Editor

To open a file for editing, use the editor's command followed by the filename:

```
$ nano filename.txt
$ vim filename.txt
$ gedit filename.txt
```

If the file does not exist, it is created automatically when you save and exit.

4.2.1 Using Nano

`nano` is a simple and beginner-friendly terminal text editor.

To start:

```
$ nano filename.txt
```

Basic Commands (shown at the bottom of the screen):

- `Ctrl + O` — Save the file (then press Enter to confirm).
- `Ctrl + X` — Exit the editor.
- `Ctrl + K` — Cut a line.
- `Ctrl + U` — Paste the cut line.
- `Ctrl + W` — Search for text.
- `Ctrl + G` — Display help.

`nano` is ideal for quick edits and configuration file changes.

4.2.2 Using Vim

`vim` is a powerful modal text editor designed for advanced editing and automation tasks.

To start:

```
$ vim filename.txt
```

Modes:

- **Normal mode** — used for navigation and commands.
- **Insert mode** — used for typing text.
- **Command mode** — used for saving and quitting.

Basic Commands:

- Press `i` — enter Insert Mode to start editing before the current character.
- Press `o` in Normal Mode — open a new line below the current line and enter Insert Mode.
- Press `G` in Normal Mode — move to the last line of the file.
- Press `Esc` — return to Normal Mode.
- Type `/word` and press Enter — search forward for `word`; press `n` to move to the next match.
- Type `:s/old/new/g` — replace every occurrence of `old` with `new` on the current line.
- Type `:%s/old/new/g` — replace every occurrence throughout the file.
- Type `:w` — save the file; `:q` — quit Vim.
- Type `:wq` — save and quit; `:q!` — quit without saving.

Example: To change every occurrence of `OPEN` to `REVIEWED`, type `:%s/OPEN/REVIEWED/g` in Command Mode and press Enter.

`vim` is efficient for editing large files, programming, and automating repetitive edits.

4.2.3 Using Gedit

`gedit` is a graphical text editor suitable for desktop environments such as Ubuntu.

To start (GUI mode):

```
$ gedit filename.txt &
```

The trailing `&` runs `gedit` in the background, so the terminal remains usable while the editor stays open.

Basic Features:

- Menu-based options for opening, saving, and editing files.
- Syntax highlighting for many programming languages.
- Ideal for users who prefer a graphical interface.

4.3 Searching in Files

Searching for specific content within text files is a common and powerful operation in Linux. The main tool for this purpose is `grep` (Global Regular Expression Print). It prints lines that match a given pattern or word, making it especially useful for analyzing log files, configuration files, and source code.

4.3.1 Basic Usage

The simplest form of the `grep` command searches for a specific word or phrase:

```
$ grep "pattern" filename.txt
```

This command prints all lines in `filename.txt` that contain the word `pattern`.

Example:

```
$ grep "error" system.log
```

This displays all lines in `system.log` that contain the word "`error`".

4.3.2 Common Options

`grep` supports several useful options that enhance its searching capabilities:

Option	Description	Example
`-i`	Ignore case differences when searching	`grep -i "error" log.txt`
`-n`	Display line numbers along with matches	`grep -n "root" /etc/passwd`
`-v`	Invert match (show lines that *do not* contain the pattern)	`grep -v "INFO" logfile.txt`
`-r`	Search recursively through all subdirectories	`grep -r "main" ./src`
`-A N`	Show `N` lines *after* each match	`grep -A 2 "ERROR" log.txt`
`-B N`	Show `N` lines *before* each match	`grep -B 2 "ERROR" log.txt`
`-C N`	Show `N` lines of context (before and after matches)	`grep -C 3 "ERROR" log.txt`

Table 4.1: Common `grep` Command Options in Linux

Searching Multiple Files:

You can search across several files at once:

```
$ grep "main" *.c
```

This command finds occurrences of "`main`" in all `.c` files in the current directory.

4.3.3 Using Regular Expressions with Grep

`grep` can use regular expressions (regex) to match complex patterns instead of exact words.

Examples:

```
$ grep "^A" names.txt        # Lines starting with A
$ grep "error$" log.txt      # Lines ending with 'error'
$ grep "[0-9]" data.txt      # Lines containing a number
$ grep -E "cat|dog" pets.txt # Lines containing 'cat' or 'dog'
```

Use `grep -E` (or `egrep`) for extended regular expressions, which support more advanced patterns such as alternation (`|`).

4.4 Text Processing Commands

Linux provides several commands for handling and analyzing text files. These commands are often used together to **filter**, **modify**, or **summarize** data.

Common Text Processing Commands:

- `cut` – extract specific columns or fields from a file.

```
$ cut -d ":" -f1 /etc/passwd    # Shows the first field in each line
```

- `tr` – translate or delete characters.

```
$ tr 'a-z' 'A-Z' < file.txt    # Converts all letters to uppercase
```

- `sort` – sort lines alphabetically or numerically.

```
$ sort names.txt
```

- `uniq` – remove duplicate lines (usually used after `sort`).

```
$ sort names.txt | uniq
```

- `wc` – count lines, words, and characters.

```
$ wc -l file.txt
```

- `awk` – print selected columns or perform basic text processing.

```
$ awk '{print $1, $3}' data.txt
```

- `sed` – search and replace text.

```
$ sed 's/error/warning/g' logfile.txt
```

These commands can be combined with pipes (`|`) for more powerful results.

Example:

```
$ cat logfile.txt | grep "ERROR" | sort | uniq -c
```

This finds all "ERROR" lines, sorts them, removes duplicates, and counts occurrences.

4.5 Redirection and Pipes

Redirection and pipes let you control where a command's input and output go. They are essential for saving results and combining commands. Figure 4.1 summarizes the three mechanisms below.

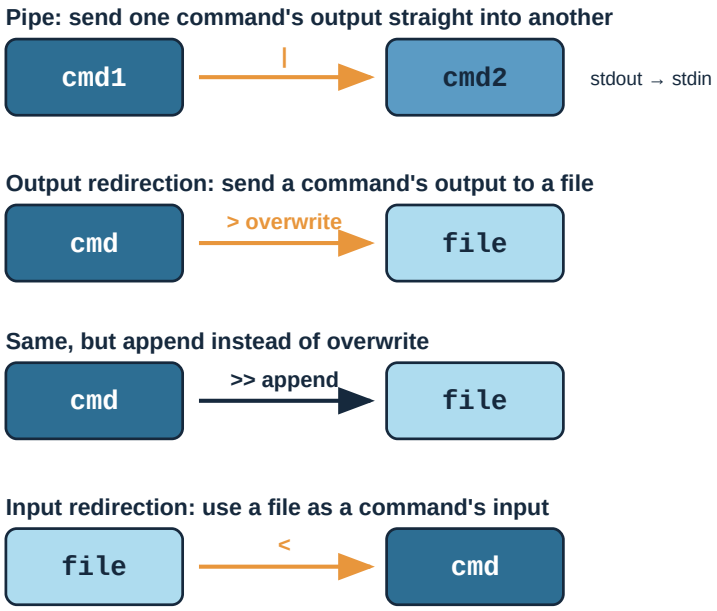


Figure 4.1: Output redirection, append, input redirection, and pipes: where data goes in each case.

4.5.1 Output Redirection

- > — write output to a file, overwriting it:

```
$ echo "Hello" > greeting.txt
```

- » — append output to a file:

```
$ echo "World" >> greeting.txt
```

4.5.2 Input Redirection

- < — use a file as input instead of the keyboard:

```
$ sort < names.txt
```

4.5.3 Pipes

- | — send the output of one command to another:

```
$ cat file.txt | grep "error"
$ ps aux | grep "python"
```

Example:

```
$ grep "ERROR" log.txt | sort | uniq -c > result.txt
```

This command:

- Finds lines with "ERROR"
- Sorts and counts them
- Saves the output to `result.txt`

4.6 Hands-on Practices

Complete the following three practices using the Linux terminal. Record the commands you use, save your edited files, and keep all original practice data unchanged. The downloadable dataset is required only for Practices 2 and 3.

4.6.1 Practice 1: Create and Edit a Text File

Work in your `os-lab/` directory. This practice does not require the downloadable dataset.

1. Create a directory named `chapter4-practice`, move into it, and create `incident-note.txt` using `>` and `»`. Include the title `INCIDENT REPORT`, the line `Status: OPEN`, the line `Service: web`, and a line describing the next action.
2. Open `incident-note.txt` with `nano`. Change `Status: OPEN` to `Status: IN PROGRESS`, add an `Owner:` line with your initials, then save and exit. Verify the changes in the terminal.
3. Open the same file with `vim`. Type `:%s/IN PROGRESS/REVIEWED/g` and press Enter to replace the status. Then press `G`, press `o`, type a new line beginning `Follow-up:`, press `Esc`, and save and quit with `:wq`.
4. Display the final file and use `wc` to count its lines, words, and characters.

In your lab notes, explain the difference between `>` and `»`, and identify one difference between editing with Nano and Vim.

4.6.2 Practice 2: Investigate a Service Log

Download and extract the practice archive, then read `README.txt`. The dataset is available here:

Download the Chapter 4 text-file data (ZIP)

Work with the extracted files and create a directory named `results/`. Do not modify the original files.

1. Inspect `service.log` using `head`, `tail`, and `wc -l`.
2. Count the lines containing `ERROR` and `WARN`; then search for `timeout` without regard to letter case and display the matching line numbers.
3. Use a pipeline with `grep`, `cut`, `sort`, and `uniq -c` to count errors by service, and save the result as `results/error-counts.txt`. Then use `nano` to create `results/log-observations.txt`, recording which service has the highest error count and one observation from the log. Save the file and verify both outputs.

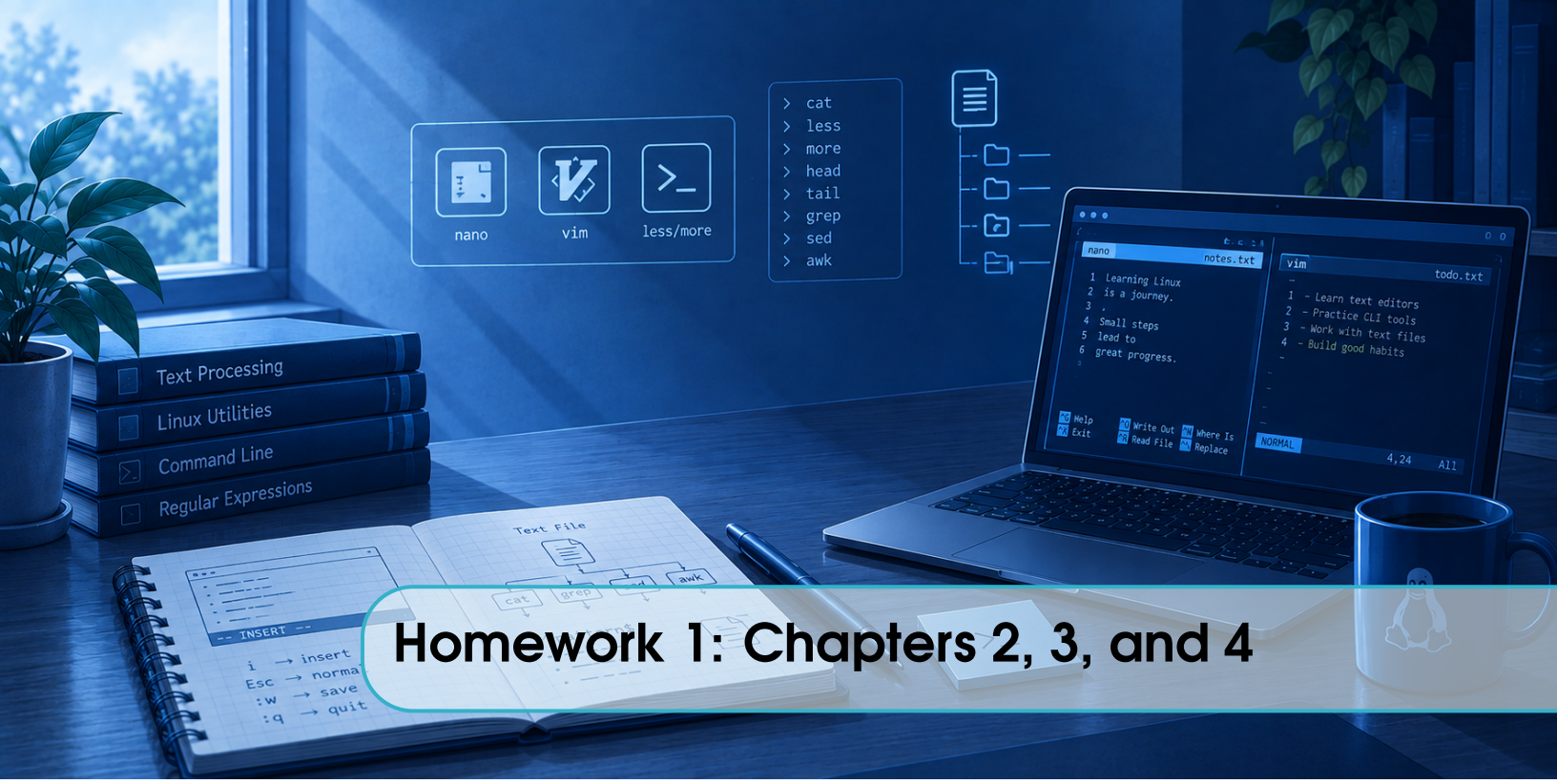
In your lab notes, explain what `grep -i` does and why `sort` is used before `uniq -c`.

4.6.3 Practice 3: Summarize and Clean Text Data

Continue using the downloaded files and save all generated files in `results/`. Keep the originals unchanged.

1. Use `tickets.csv` to count tickets by priority and by status, excluding the header row. Save both summaries, with clear headings, in `results/ticket-summary.txt`.
2. Use `sed` to replace uppercase `TEMP` with `ARCHIVED` in `raw_notes.txt`, saving the result as `results/cleaned-notes.txt`. Then convert the words in `words.txt` to lowercase, sort them, count repeated words, and save the result as `results/word-counts.txt`.
3. Use `nano` to create `results/incident-summary.txt`. Include the exact phrase `most common status`, the service with the highest error count, and one limitation of drawing conclusions from these sample files. Save and exit Nano, then open the report in `vim` and type `:%s/most common status/most frequent status/g` to replace that phrase throughout the report. Save, quit, and display the final report.

Be prepared to change one search in your command sequence (for example, from `ERROR` to `WARN`) and explain how the result changes.



Homework 1: Chapters 2, 3, and 4

Scenario

A small operations team needs a Linux workspace for investigating a fictional service incident. The workspace must be organized, shared safely between team members, and used to turn raw text records into a short incident report. Your task is to build and verify the complete workflow using the concepts from Chapters 2, 3, and 4.

This is a take-home project, not a guided practice. Choose appropriate commands from the chapters, record the commands you use, and explain the results. Do not simply submit a list of commands without evidence that they worked.

Environment and Safety Requirements

- Complete the assignment inside your own Linux virtual machine. The tasks create system users and groups and require administrative privileges. Do not perform them on your primary computer's host operating system, a shared university server, or any machine you do not administer.
- You will need `sudo`. The ACL tasks also require the `getfacl` and `setfacl` utilities; install the relevant ACL package in your VM if these commands are unavailable.
- Use the names `hw_handover`, `hw_writer`, `hw_auditor`, and `hw_guest` throughout the assignment. Check that they do not already exist. If a name is already in use, add the same short suffix to the affected name and use that replacement consistently.
- Do not delete or modify system files such as `/etc/passwd` or `/etc/shadow` manually. Use the appropriate Linux administration commands.

Create and Organize the Workspace

Create the project directory `/srv/handover-lab` with the following structure:

```
/srv/handover-lab/
|-- incoming/
|   |-- events.log
|   |-- operator-notes.txt
|   ‘-- backup/
|-- reports/
|-- dropbox/
|-- private/
|-- public/
‘-- .handover-policy
```

Complete the following requirements:

1. Create the directories and files. Put the supplied event records listed below into `incoming/events.log`. Create `operator-notes.txt` using Nano and enter the note text listed below.
2. Create the hidden file `.handover-policy`. It must contain three lines describing the retention period, the person responsible for preparing reports, and the person responsible for reviewing them.
3. Make a backup copy of `events.log` in `incoming/backup/`. Keep the original file unchanged during the later analysis.
4. Use ordinary and detailed directory listings to verify the structure. Demonstrate why the hidden policy file is absent from an ordinary listing but visible when hidden files are included.
5. Record the filesystem type and mount information for the filesystem containing `/srv/handover-lab`. Include the relevant output from `df -Th` in your evidence.

Configure Team Accounts and Access

The workspace has three roles:

- `hw_writer`: creates and updates incident material.
- `hw_auditor`: reviews reports and may read the restricted incident details, but must not modify them.
- `hw_guest`: may read the public summary only; this account must not be a member of `hw_handover`.

Configure the accounts, group, directories, and files so that all of the following conditions are true. You must select the appropriate commands and permission values yourself.

1. Create the group `hw_handover` and the three user accounts, each with a home directory and Bash as the login shell. Add the writer and auditor to `hw_handover`; do not add the guest to that group. Verify the memberships using account and group inspection commands.
2. Set group ownership and permissions on `incoming/` and `reports/` so that the writer and auditor can access and modify their contents, while unrelated users have no access. New files and subdirectories created there must inherit the project group. For collaborative files, ensure the group has write permission where required; the SGID bit does not automatically make files group-writable. Create a test file as the writer and demonstrate that the auditor can update it.

3. Configure `dropbox/` as a shared drop-off directory. Both writer and auditor must be able to create files in it, new items must inherit the project group, and one team member must not be able to delete or rename another team member's files merely because they share the directory.
4. Configure `public/` so that the guest can read `public/summary.txt` but cannot modify it. The guest must not be able to write to the other project directories.
5. Configure `private/` so that `hw_writer` owns and can modify `private/incident-details.txt`. Create this file as the writer and include a short fictional internal note based on the log; do not include real personal information. The auditor must be able to read that file but must not be able to modify it. The guest must be denied access. Use an ACL for the auditor's file-level permission and verify it with `getfacl`.
6. Verify the behavior by attempting the relevant operations as the specified accounts. In particular, demonstrate that the auditor can read but cannot edit the restricted file, that the guest is denied access to it, and that the guest can read but cannot write the public summary. Also test the shared-directory deletion rule with a file created by the writer and a deletion attempt by the auditor.

Your evidence must include the relevant output from `id` or equivalent account-inspection commands, detailed permission listings, the ACL for the restricted file, and the results of the access tests. A permission listing alone is not enough; verify actual access as each role.

Investigate the Event Log and Process Text

Create `incoming/events.log` with the following 15 fictional records. Keep the field order unchanged: date, time, level, service, and message, separated by the vertical-bar character.

```
2026-10-20|09:00:02|INFO|auth|login accepted for user=ali
2026-10-20|09:02:14|WARN|gateway|response delay reached 1.8 seconds
2026-10-20|09:03:05|ERROR|storage|write failed on volume=v2
2026-10-20|09:04:33|INFO|scheduler|job queued id=H100
2026-10-20|09:07:19|ERROR|gateway|upstream connection refused
2026-10-20|09:08:41|WARN|auth|repeated login attempt detected
2026-10-20|09:09:12|ERROR|storage|read failed on block=7
2026-10-20|09:12:06|INFO|network|link state stable
2026-10-20|09:14:28|ERROR|auth|token validation failed
2026-10-20|09:16:53|WARN|storage|disk usage reached 80 percent
2026-10-20|09:19:07|ERROR|gateway|request timeout after 4 seconds
2026-10-20|09:22:31|ERROR|scheduler|job H104 failed with code 5
2026-10-20|09:24:09|WARN|gateway|response delay reached 2.1 seconds
2026-10-20|09:28:44|ERROR|auth|token validation failed
2026-10-20|09:30:10|INFO|storage|health check completed
```

Create `incoming/operator-notes.txt` with Nano and the following content:

```
NIGHT SHIFT HANDOVER
STATUS: OPEN
Temporary note: TEMP cache cleared after gateway timeout.
STATUS: pending review
Temporary note: TEMP diagnostic files must be retained.
```

```
Action: review gateway latency and storage errors.
```

Then complete these tasks:

1. Inspect the event log using suitable viewing and counting commands. Search for all error records, count the warning records, and search for the timeout message without relying on its letter case. Preserve the matching output in files under `reports/`.
2. Build a pipeline that extracts the service field from error records, groups identical service names, counts them, and saves the summary as `reports/errors-by-service.txt`. Your pipeline must use `grep`, `cut`, `sort`, and `uniq`. Do not manually type the counts into the result file.
3. Use `sed` to replace each occurrence of the uppercase text `TEMP` with `ARCHIVED` in the operator notes. Save the transformed result as `reports/cleaned-notes.txt`; do not alter the original notes.
4. Use redirection to save the records containing `ERROR` as `reports/errors.log`. Use `wc` to record the number of lines in the original event log and the extracted error file.

Write, Review, and Publish the Handover Report

Use Nano to create `reports/incident-report.txt`. The report must contain:

- A concise statement of the incident being investigated.
- The number of error and warning records found.
- The service or services with the highest error count, supported by the generated summary file.
- The timeout evidence and the relevant log line number.
- One limitation of drawing conclusions from a small, fictional log sample.

Next, open the report in Vim and make a meaningful revision after checking your own analysis. The revision must be visible in the final file; record which Vim operation you used. Save the final report and verify its contents in the terminal.

Create `public/summary.txt` as a short version of the incident report. It may contain general findings, but it must not contain the restricted incident details. Set its permissions so the guest can read it but cannot change it. Move or rename any temporary report draft as appropriate so that the final report has a clear, unambiguous filename.

Explanation and Submission Checklist

Answer each question in your own words:

1. Explain the difference between the SGID bit and the Sticky Bit on a shared directory. Which requirement in this assignment depends on each one?
2. Why is an ACL appropriate for granting the auditor read-only access to the restricted file? Why is membership in the project group alone not sufficient to meet the private-directory requirement?

3. Explain the difference between > and », and why `sort` is normally used before `uniq -c` in the service-counting pipeline.
4. Explain the difference between read and execute permission on a directory. Refer to one access test from your workspace.

Submit one ZIP archive containing:

- The final project files and generated analysis files, including `errors.log`, `errors-by-service.txt`, `cleaned-notes.txt`, `incident-report.txt`, and `public/summary.txt`.
- A plain-text record of the commands you used. Include commands that show the final ownership, permissions, and ACL; do not include passwords or other secrets.
- Evidence of the successful and denied access tests for the writer, auditor, and guest, plus the output of `df -Th` for the workspace filesystem.
- Your answers to the four explanation questions.

Do not submit only screenshots or only command text. Include the relevant outputs and final files so the results can be checked. Do not remove the test accounts or workspace until your instructor has confirmed that the submission is complete.

5. Shell Programming I

Shell scripting is a fundamental skill for system administrators and advanced Linux users. It lets you automate repetitive tasks, manage system operations, and build powerful tools simply by combining the commands you already know from Chapter 4 and Chapter 2.

5.1 What is a Shell?

The shell is a command-line interface (CLI) that acts as an intermediary between the user and the operating system kernel. It provides several key capabilities:

- Interactive command execution for file navigation and process management.
- Scripting capabilities that allow automation of complex tasks.
- Command piping and redirection for combining multiple operations.

The shell is the outermost layer surrounding the operating system: it interprets the commands and scripts you type and translates them into requests the kernel can act on. Figure 5.1 shows where it sits relative to the kernel and the hardware.

5.2 Types of Shells

Linux systems support multiple shell implementations, each with unique features:

- `bash` — Bourne Again Shell, the default on most modern Linux systems.
- `sh` — the original Bourne shell, predecessor to `bash`.
- `zsh`, `fish`, `csh` — alternative shells offering additional features and customizations.

All available shell interpreters are listed in the `/etc/shells` file.

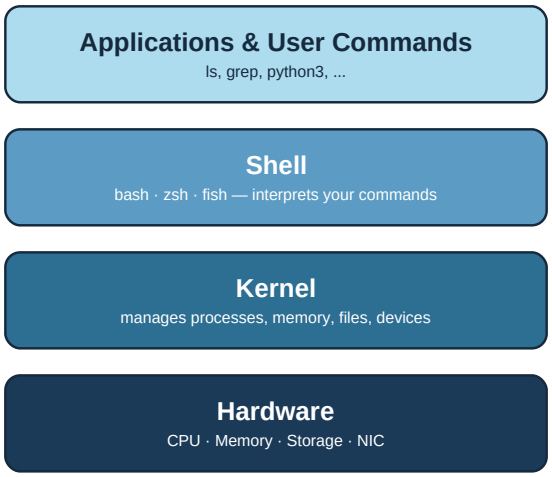


Figure 5.1: The shell's place in the system: it sits between the applications you run and the kernel that manages the hardware.

5.3 Shell Interpreter Working Modes

Shell interpreters operate in two distinct modes:

- **Interactive mode** — the interpreter interacts directly with the user, accepting commands and displaying results in real time.
- **Non-interactive mode** — the interpreter executes commands from a script file without user interaction.

How to Run a Shell Script:

1. Open a text editor and write your shell commands. Save the file with a `.sh` extension (for example, `myscript.sh`).
2. Run the script, using either method:

```
$ bash myscript.sh
```

or make it executable and run it directly:

```
$ chmod +x myscript.sh
$ ./myscript.sh
```

5.4 Shell Scripts

A shell script is a text file containing a series of shell commands executed sequentially by the shell interpreter.

Basic Script Structure:

```
#!/bin/bash
# My first script
echo "Hello, $USER"
echo "Today is $(date)"
```

The first line `#!/bin/bash` (called a *shebang*) tells the OS to use `bash` as the interpreter. Lines beginning with `#` are comments. The script uses the variable `$USER` and command substitution `$(date)` to display dynamic information. Figure 5.2 breaks this example down line by line.

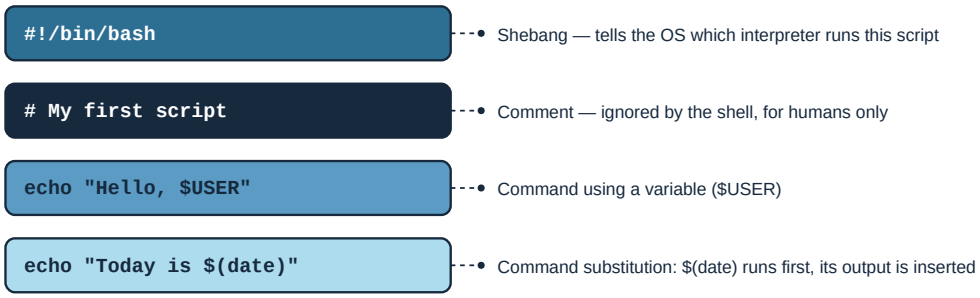


Figure 5.2: Anatomy of a simple shell script.

5.5 Variables

Variables store data values that can be referenced and manipulated throughout a script.

Basic Assignment:

```
#!/bin/bash
variable_name=value
```

Important: No spaces are allowed around the equals sign; spaces act as delimiters in the shell.

Accessing Variables:

Use the $ prefix to retrieve a variable's value:

```
#!/bin/bash
x=4
echo $x          # Outputs: 4
echo $undefined  # Outputs: empty string
```

Undefined variables produce empty strings rather than errors.

Performing Arithmetic:

By default, the shell treats all values as strings. To perform arithmetic operations, use one of these methods:

1. Using `(( ))`:

```
#!/bin/bash
```

```
x=4
((y = x + 1))
echo $y  # Outputs: 5
```

2. Using `let`:

```
#!/bin/bash
x=4
let y=x+1
echo $y  # Outputs: 5
```

3. Using `declare` with the `-i` flag:

```
#!/bin/bash
x=4
declare -i y=x+1
echo $y  # Outputs: 5
```

5.6 The `declare` Command

The `declare` command lets you create variables and set their attributes:

```
declare [options] variable_name
```

Common Options:

Option	Meaning
`-a`	Create an indexed array
`-i`	Create an integer variable
`-l`	Convert variable to lowercase
`-u`	Convert variable to uppercase
`-r`	Create a read-only variable
`-g`	Create a global variable

Table 5.1: Common Shell Variable Options in Linux

Examples:

```
declare -i count=5       # Integer variable
declare -u name="john"   # Uppercase: NAME="JOHN"
declare -l text="HELLO"  # Lowercase: text="hello"
declare -r constant=100  # Read-only variable
```

5.7 Arrays

An array is a collection of values stored in a single variable. Instead of creating separate variables for each item, you can group related values together in an array. Each value in the array has a position number called an *index*, starting from 0.

5.7.1 Creating Arrays

1. Assign elements one at a time:

```
colors[0]="red"
colors[1]="green"
colors[2]="blue"
```

2. Create an array with multiple elements at once:

```
colors=("red" "green" "blue")
numbers=(10 20 30 40)
```

3. Add more elements to an existing array:

```
colors=("red" "green" "blue")
colors+=("yellow")  # Adds yellow to the end
```

5.7.2 Accessing Array Elements

View all elements in an array:

```
echo ${colors[*]}
echo ${colors[@]}
# Both output: red green blue yellow
```

Access a specific element by its position (index):

```
echo ${colors[0]}  # Outputs: red
echo ${colors[1]}  # Outputs: green
echo ${numbers[2]} # Outputs: 30
```

Creating Arrays with Automatic Values:

Use `{start..end}` to create a series automatically:

```
letters={a..e}
echo ${letters[*]}  # Outputs: a b c d e

numbers={1..5}
echo ${numbers[*]}  # Outputs: 1 2 3 4 5
```

Using the `seq` Command for Number Sequences:

The `seq` command generates a sequence of numbers with a specific step:

```
# Create array from 10 to 50, counting by 10
numbers=($(seq 10 10 50))
echo ${numbers[*]}  # Outputs: 10 20 30 40 50

# Create array from 100 down to 20, counting by -5
countdown=($(seq 100 -5 20))
echo ${countdown[0]}  # Outputs: 100
```

5.8 Conditional Statements

Conditional statements allow your script to make decisions based on conditions.

5.8.1 The `test` Command

The `test` command evaluates an expression and sets the exit status. Use either `test` or `[ ]`:

```
test "abcd" = "abcd"
echo $?              # Outputs: 0 (true)

[ "abcd" = "abcd" ]
echo $?              # Outputs: 0 (true)
```

In bash, 0 represents `true` and any non-zero value represents `false`.

5.8.2 Combining Conditions

Use `&&` (AND) and `||` (OR) to combine conditions:

```
[ "abcd" = "abcd" ] && [ 3 -gt 2 ]
echo $?  # Outputs: 0 (both conditions true)

[[ "abcd" = "abcd" && 3 -gt 2 ]]
echo $?  # Same result using [[ ]]
```

Note: `[[ ]]` is an extended bash feature that provides more functionality than `[ ]`.

5.8.3 The `if` Command

The `if` command is a control structure that executes code conditionally:

```
if condition ; then
    # commands executed if condition is true
elif condition ; then
    # commands executed if the previous condition was false
else
    # commands executed if all conditions are false
fi
```

The condition can be a `test` command (`[ ]` or `[[ ]]`), an arithmetic expression `(( ))`, or any command with an exit status.

Examples:
File existence check:

```
if [ -e a.txt ]; then
    echo "The file exists"
else
    echo "File not found"
fi
```

Arithmetic comparison:

```
if ((2 + 3 == 5)); then
    echo "True"
else
    echo "False"
fi
```

5.9 Common Test Operators

5.9.1 File Tests

- `-e file` — file exists
- `-f file` — file is a regular file
- `-d dir` — directory exists
- `-r file` — file is readable
- `-w file` — file is writable
- `-x file` — file is executable

5.9.2 String Tests

- `str1 = str2` — strings are equal
- `str1 != str2` — strings are not equal
- `-z str` — string is empty
- `-n str` — string is not empty

5.9.3 Numeric Tests

- `n1 -eq n2` — numbers are equal
- `n1 -ne n2` — numbers are not equal
- `n1 -lt n2` — `n1` is less than `n2`
- `n1 -gt n2` — `n1` is greater than `n2`
- `n1 -le n2` — `n1` is less than or equal to `n2`
- `n1 -ge n2` — `n1` is greater than or equal to `n2`

5.10 Hands-on Practices

Complete the following three practices using the Linux terminal. Record the commands you use, save your scripts, and explain the results where requested. Work inside your `os-lab/` directory.

5.10.1 Practice 1: Your First Scripts, Variables, and Arithmetic

This practice covers shells, running scripts, variables, arithmetic, and the `declare` command.

1. Create a directory named `chapter5-practice` and move into it. Display the contents of `/etc/shells` and find out which shell you are currently using.
2. Create a script named `info.sh` that starts with a shebang line, contains one comment, and prints a greeting using `$USER` and the current date using `$(date)`. Run it first with `bash info.sh`, then make it executable with `chmod` and run it as `./info.sh`.
3. Create a second script named `vars.sh` that does the following:
   - assigns `x=4` and prints it, and also prints a variable that was never defined;
   - calculates `y = x + 1` three times, once with `(( ))`, once with `let`, and once with `declare -i`, and prints each result;
   - creates an uppercase variable (`declare -u`), a lowercase variable (`declare -l`), and a read-only variable (`declare -r`), prints them, and then tries to change the read-only variable.

Now, answer the following questions:

- What is the difference between interactive and non-interactive mode? Which mode was used when you ran `info.sh`?
- What is the purpose of the shebang line, and why are spaces not allowed around the = sign in an assignment?
- What did the undefined variable print, and what happened when you tried to modify the read-only variable?

5.10.2 Practice 2: Working with Arrays

This practice covers creating, extending, and accessing arrays.

1. Create a script named `arrays.sh`. In it, create an array `colors` by assigning the elements `red`, `green`, and `blue` one at a time using their indexes. Create an array `numbers` containing `10 20 30 40` in a single assignment, and then add `yellow` to the end of `colors` using `+=`.
2. Print all elements of `colors` using both `${colors[*]}` and `${colors[@]}`, then print the first element of `colors` and the third element of `numbers`.
3. Create the array `letters` using `{a..e}`, an array `evens` from 10 to 50 in steps of 10 using `seq`, and an array `countdown` from 100 down to 20 in steps of -5 using `seq`. Print all of them, and use `(( ))` to calculate and print the sum of the first and last elements of `evens`.

Now, answer the following questions:

- At which index does an array start, and what does `colors+=("yellow")` do?
- What is the difference between creating a sequence with `{1..5}` and with `seq`? Which one allows a custom step?

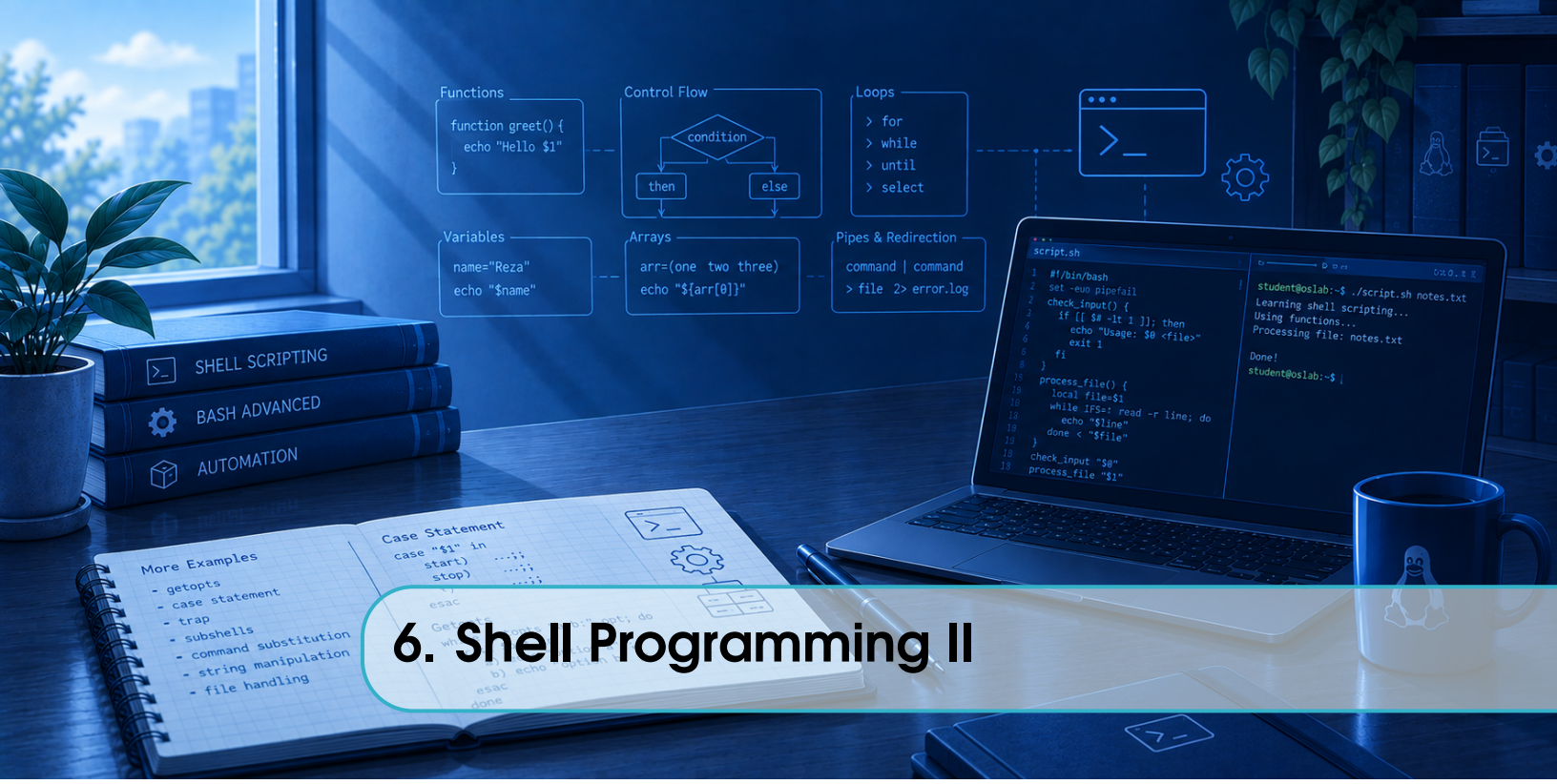
5.10.3 Practice 3: Conditions and a File Checker Script

This practice covers `test`, exit status, combined conditions, `if/elif/else`, and the file, string, and numeric test operators.

1. In the terminal, evaluate a true and a false string comparison using `test` and `[ ]`, and print the exit status with `echo $?` after each. Then combine `[ "abcd" = "abcd" ]` with `[ 3 -gt 2 ]` using `&&`, combine a false condition with a true one using `||`, and repeat the first combination using `[[ ]]`.
2. Create a script named `check.sh` that stores a file name in a variable (for example, `file="data1.txt"`) and uses `if`, `elif`, and `else` to report whether the path does not exist, is a directory, or is a regular file. For a regular file, also report whether it is readable, writable, and executable. Run the script on an existing file, on a directory, and on a missing path by changing the variable.
3. Extend `check.sh` for regular files. Store the number of lines of the file in a variable (for example, `lines=$(wc -l < "$file")`), and use numeric test operators to classify the file as empty, short (up to 10 lines), or long. Then use `-z` and `-n` to check whether a string variable is empty, and use an arithmetic condition with `(( ))` to check whether `lines` is even or odd.

Now, answer the following questions:

- Which exit status represents true in bash, and which represents false?
- What is the difference between `[ ]` and `[[ ]]`?
- Why must you use `-eq` instead of = when comparing numbers with `[ ]`?



6. Shell Programming II

This chapter builds directly on Chapter 5: with variables, conditionals, and test operators already in hand, the missing piece for writing genuinely useful scripts is **repetition** – running the same commands over a list of items, or until some condition changes. That is what loops give you.

6.1 Introduction to Loops

Loops let you repeat commands in a script, which is useful for handling repetitive tasks efficiently. Instead of writing the same code many times, a loop runs it automatically over a list or for as long as a condition holds.

6.1.1 Why Use Loops

In system administration and scripting, loops give you three things at once:

- **Efficiency:** automate repetitive work, such as processing many files or counting through a range of numbers.
- **Readability:** keep scripts short and easy to maintain, instead of repeating near-identical lines.
- **Common uses:** repeating an action (e.g. printing messages), processing data (e.g. reading lines from a file), or waiting for a condition (e.g. until a counter reaches zero).

Without loops, you would need to write the same commands multiple times, making scripts longer, harder to maintain, and more prone to errors.

6.1.2 Types of Loops in Bash

Bash provides three primary loop structures, each suited to a different scenario:

6.1.3 Loop Execution Flow

Regardless of which loop you use, the same four steps repeat in a cycle, shown in Figure 6.1:

Loop Type	Best Used When
`for`	You have a specific list of items to process
`while`	You want to continue *while* a condition remains true
`until`	You want to continue *until* a condition becomes true

Table 6.1: Common Loop Types in Shell Scripting

1. **Initialization:** set up any variables the loop depends on.
2. **Condition Check:** evaluate whether to continue.
3. **Body Execution:** run the commands inside the loop.
4. **Update:** modify the loop variable, then return to step 2.

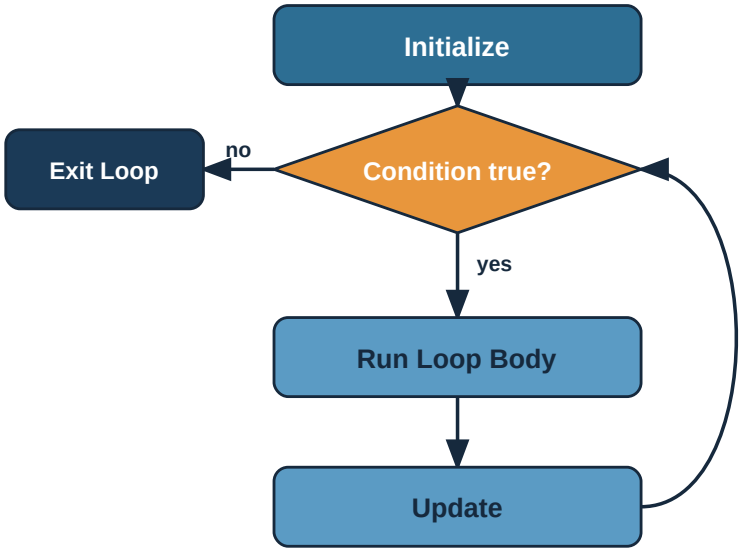


Figure 6.1: The general execution flow shared by `for`, `while`, and `until` loops.

6.2 The `for` Loop

The `for` loop is great for going through a fixed list of items, such as numbers or words. It assigns each item to a variable one at a time and runs the loop body once per item.

6.2.1 Basic Syntax and Explanation

```
#!/bin/bash
for variable in list; do
    # Commands to run for each item
done
```

- **variable**: holds the current item from the list on each pass.
- **list**: the items to loop through (e.g. words or numbers).
- **do ... done**: the block of commands that runs on every pass.

This loop runs exactly once per item in the list – there is no separate condition check like in `while/until`.

6.2.2 Creating Lists for Loops

You can build the list in several simple ways:

- Direct words: `apple banana orange`
- Numbers with brace expansion: `{1..5}` (expands to `1 2 3 4 5`)
- From a command's output: `$(ls *.txt)` (lists all `.txt` files)

6.2.3 Practical Basic Examples

Example 1: Print Fruits

This loops through a list of fruits and prints a message for each.

```
#!/bin/bash
for fruit in apple banana orange; do
    echo "I like $fruit"
done
```

Explanation: the list is `apple banana orange`; for each one, the script echoes `"I like [fruit]"`.

```
I like apple
I like banana
I like orange
```

Example 2: Count Numbers

This uses brace expansion for numbers.

```
#!/bin/bash
for num in {1..5}; do
    echo "Number: $num"
done
```

Explanation: `{1..5}` creates the list `1 2 3 4 5`, and the loop prints each number.

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

Useful for fixed ranges known ahead of time.

6.2.4 C-style `for` Loop

Bash also supports a C-style `for` loop syntax, which is convenient for numeric iteration:

```
#!/bin/bash
for ((initialization; condition; update)); do
    # commands
done
```

Example:

```
#!/bin/bash
for ((i=0; i<10; i++)); do
    echo "Iteration: $i"
done

# With step increments
for ((i=0; i<=20; i+=5)); do
    echo "Value: $i"
done
```

6.3 The `while` Loop

The `while` loop repeats commands for as long as a condition is true. It's the right choice when you don't know in advance how many times the loop needs to run.

6.3.1 Basic Syntax and Explanation

```
#!/bin/bash
while [ condition ]; do
    # Commands
done
```

- `[ condition ]` is checked before each pass (e.g. `[ $counter -gt 0 ]`).
- If it is true (exit status 0), the loop body runs; otherwise, the loop stops.
- Remember to update the tested variable inside the loop body – forgetting to do so causes an infinite loop.

6.3.2 Reading Files with `while`

A common use of `while` is reading a file line by line:

```
#!/bin/bash
while read line; do
    echo "$line"
done < file.txt
```

This reads each line of `file.txt` into `$line` and processes it, one line per pass.

6.3.3 Practical Basic Examples

Example 1: Countdown
This counts down from 5 to 1.

```
#!/bin/bash
counter=5
while [ $counter -gt 0 ]; do
    echo "Countdown: $counter"
    ((counter--))  # Decrease counter by 1
done
```

Explanation: starts with `counter=5`; the condition is "while `counter > 0`"; the script prints and decreases the counter on every pass.

```
Countdown: 5
Countdown: 4
Countdown: 3
Countdown: 2
Countdown: 1
```

This demonstrates both condition checking and updating in the same loop.

Example 2: Sum Numbers
This sums the numbers from 1 to 5.

```
#!/bin/bash
sum=0
num=1
while [ $num -le 5 ]; do
    sum=$((sum + num))
    ((num++))
done
echo "Sum: $sum"
```

Explanation: initializes `sum=0` and `num=1`, then adds `num` to `sum` while `num <= 5`, increasing `num` each pass. Output: `Sum: 15`. This shows how arithmetic accumulates across loop iterations.

6.4 The `until` Loop

The `until` loop is the mirror image of `while`: it keeps running *as long as* its condition is false, and stops as soon as the condition becomes true.

6.4.1 Basic Syntax and Explanation

```
#!/bin/bash
until [ condition ]; do
    # Commands
done
```

- The loop body runs while the condition is false (non-zero exit status).
- `until [ condition ]` is equivalent to `while ! [ condition ]`.

6.4.2 Comparison with `while`

The two loops below behave identically – they are just two ways of expressing the same stopping point:

```
while [ $x -lt 5 ]; do ... done    # keep going while x < 5
until  [ $x -ge 5 ]; do ... done   # keep going until x >= 5
```

Use `until` when it is more natural to think in terms of "stop once this happens" rather than "keep going while this holds".

6.4.3 Practical Basic Examples

Example 1: Count Up

This counts from 1 to 5.

```
#!/bin/bash
counter=1
until [ $counter -gt 5 ]; do
    echo "Counter: $counter"
    ((counter++))
done
```

Explanation: starts at `1`; runs until `counter > 5` (i.e. while `counter <= 5`); prints and increases the counter each pass.

```
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
```

Example 2: Wait for a Number

This waits until a variable reaches 3.

```
#!/bin/bash
attempt=0
until [ $attempt -eq 3 ]; do
    ((attempt++))
    echo "Attempt: $attempt"
done
```

Explanation: increases `attempt` until it equals 3.

```
Attempt: 1
Attempt: 2
Attempt: 3
```

A simple pattern for simulating retries.

6.5 Loop Control Statements

These statements change how a loop runs – stopping it early or skipping a step – and work inside any of the three loop types.

6.5.1 The `break` Statement

`break` exits the loop immediately.

```
#!/bin/bash
for num in {1..5}; do
    if [ $num -eq 3 ]; then
        break  # Stop at 3
    fi
    echo "Number: $num"
done
```

Explanation: loops through 1–5, but exits entirely as soon as `num` reaches 3.

```
Number: 1
Number: 2
```

Useful for stopping early once you've found what you need.

6.5.2 The `continue` Statement

`continue` skips the rest of the *current* iteration and moves on to the next one – unlike `break`, it does not exit the loop.

```
#!/bin/bash
for num in {1..5}; do
    if [ $num -eq 3 ]; then
        continue  # Skip 3
    fi
    echo "Number: $num"
done
```

Explanation: skips the `echo` only when `num=3`; every other number still prints.

```
Number: 1
Number: 2
Number: 4
Number: 5
```

6.5.3 Nested Loops

A loop can contain another loop inside its body – useful for grids, tables, or any two-dimensional pattern.

```
#!/bin/bash
for i in {1..2}; do
    echo "Row $i:"
    for j in {1..3}; do
        echo "  Column $j"
    done
done
```

Explanation: the outer loop runs 2 rows; for each row, the inner loop runs all 3 columns before the outer loop advances.

```
Row 1:
  Column 1
  Column 2
  Column 3
Row 2:
  Column 1
  Column 2
  Column 3
```

6.6 Advanced Loop Techniques

These build on the loop basics above for more flexibility in real scripts.

6.6.1 Processing Arrays with Loops

A `for` loop can iterate directly over a list of values, including the array-like variables from Chapter 5:

```
#!/bin/bash
colors="red green blue"
for color in $colors; do
    echo "Color: $color"
done
```

6.6.2 Command Substitution in Loops

A command's output can become the loop's list directly:

```
#!/bin/bash
for file in $(ls); do
    echo "Item: $file"
done
```

Note: this pattern breaks on filenames containing spaces; for real file-processing scripts, prefer `for file in *` or a `while read` loop instead.

6.6.3 Loop Performance Tips

- Avoid running slow external commands inside large loops – each iteration pays their full startup cost.
- Test loop logic on a small amount of data before running it against a large dataset.

6.7 Hands-on Practices

Complete the following three practices using the Linux terminal. Record the commands you use, save your scripts, and explain the results where requested. Work inside your `os-lab/` directory.

6.7.1 Practice 1: Iterating with `for` Loops

This practice covers word lists, brace expansion, the C-style `for` loop, lists built from variables and command output, and nested loops.

1. Create a directory named `chapter6-practice` and move into it. Create a script named `for-loops.sh` that contains three loops: one that prints `I like $fruit` for the words `apple banana orange`, one that prints `Number: $num` for `{1..5}`, and one C-style loop that prints `Value: $i` from 0 to 20 in steps of 5. Make the script executable and run it.
2. Create three files named `a.txt`, `b.txt`, and `c.txt` with different numbers of lines. Create a script named `list-loops.sh` that loops over the string `"red green blue"` stored in a variable and prints each color, and then loops over `*.txt` and prints each file name together with its number of lines. Finally, add a loop over `$(ls)` that prints `Item: $file`, and run the script.
3. Create a script named `table.sh` that uses nested `for` loops to print a multiplication table for the numbers 1 to 3. For each value of the outer variable `i`, print a `Row $i:` heading, and for each value of the inner variable `j`, print a line such as `2 x 3 = 6`, calculated with `$((i * j))`.

Now, answer the following questions:

- What list does `{1..5}` expand to, and how many times does a `for` loop run for a list of n items?
- How many times did the inner loop body run in `table.sh` in total, and why?
- Why is `for file in $(ls)` not safe for real file processing, and what should you use instead?

6.7.2 Practice 2: Repeating with `while` and `until`

This practice covers condition-controlled loops, accumulating values, reading a file line by line, and the relationship between `while` and `until`.

1. Create a script named `while-loops.sh` that contains two `while` loops: a countdown that prints `Countdown: $counter` from 5 down to 1 using `((counter--))`, and a loop that adds the numbers 1 to 5 into a variable `sum` and prints `Sum: $sum` after the loop ends.

2. Create a file named `names.txt` with at least four lines, each containing a different name. Create a script named `read-file.sh` that reads `names.txt` line by line using `while read line` and prints each line together with its line number, using a counter variable that you increase on every pass.
3. Create a script named `until-loops.sh` that contains two `until` loops: one that counts from 1 to 5 and prints `Counter: $counter`, and one that increases a variable `attempt` and prints `Attempt: $attempt` until `attempt` equals 3. Then rewrite the first loop as a `while` loop that produces exactly the same output, and compare the two outputs.

Now, answer the following questions:

- What would happen if you removed the `((counter-))` line from the countdown loop, and how can you stop such a loop from the terminal?
- How is the file `names.txt` given to the `while read` loop, and when does the loop stop?
- How did you express the same stopping condition with `until` and with `while`? When is `until` more natural to use?

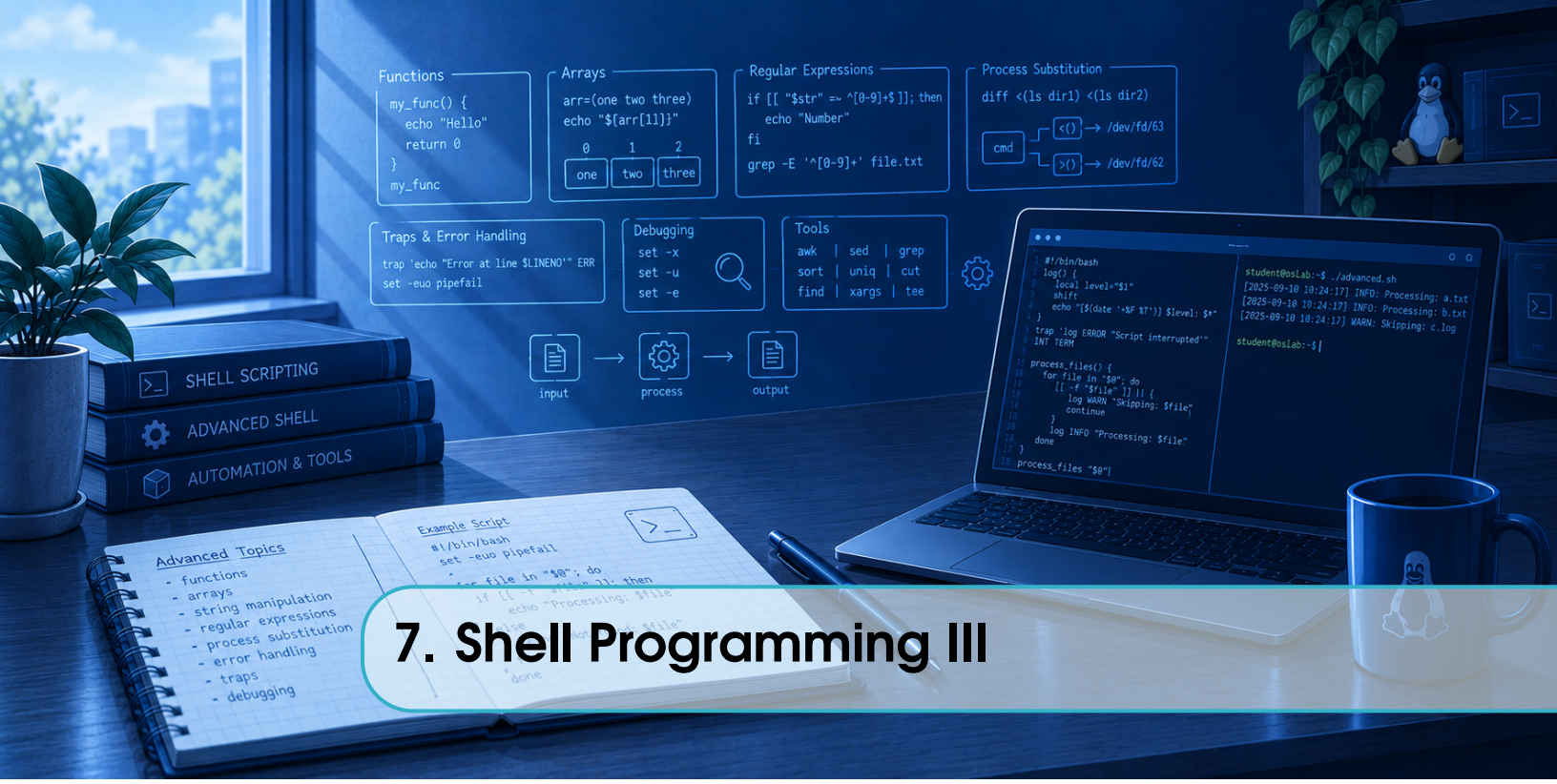
6.7.3 Practice 3: Controlling Loops and Combining Concepts

This practice covers `break` and `continue`, and combines loops with the conditions and arithmetic from Chapter 5.

1. Create a script named `control.sh` with a `for` loop over `{1..10}` that skips every number divisible by 3 using `continue` and stops the loop completely with `break` when the number becomes greater than 8. Print each remaining number as `Number: $num` and run the script.
2. Create a script named `inspect.sh` that loops over every item in the current directory using `for item in *`. For each item, use `if/elif/else` to print whether it is a directory, an empty regular file, or a non-empty regular file, using `wc -l` to count the lines of a regular file. Run the script in the `chapter6-practice` directory.
3. Create a script named `numbers.sh` with a C-style `for` loop from 1 to 20. Inside the loop, add every even number to a variable `even_sum` and count every odd number in a variable `odd_count`. Use `break` to leave the loop as soon as `even_sum` becomes greater than 50. After the loop, print the final values of `even_sum`, `odd_count`, and the last value of the loop variable.

Now, answer the following questions:

- What is the difference between `break` and `continue`? Which numbers did each of them affect in `control.sh`?
- In `numbers.sh`, at which value of the loop variable did the loop stop, and what were the final values of `even_sum` and `odd_count`?
- Why is it a good idea to test a loop on a small amount of data before running it on a large dataset?



7. Shell Programming III

With variables, conditionals, and loops from Chapters 5 and 6, you already have everything needed to write working scripts. The last piece is **functions** – a way to package a block of logic under a name, so it can be reused instead of copy-pasted, and so a long script can be organized into clear, testable pieces.

7.1 What Is a Function?

A function is a named block of code that performs a specific task. It allows you to reuse logic, reduce redundancy, and organize scripts into smaller, manageable parts.

Advantages:

- Reuse code instead of rewriting it.
- Simplify debugging and maintenance.
- Improve script readability and structure.

7.2 Defining Functions

Functions in Bash can be declared in two equivalent ways:

```
#!/bin/bash
# With the function keyword
function greet() {
    echo "Hello, $1!"
}

# Without the function keyword
greet() {
    echo "Hello, $1!"
}
```

There is **no need** to declare argument types or the number of arguments – unlike C or Python, Bash functions accept any number of arguments of any type (everything is treated as text).

7.3 Calling Functions

To execute a function, call it by its name followed by any arguments, exactly as you would call any other command:

```
greet "Ali"
```

Syntax:

```
function_name arg1 arg2 ...
```

Important: a function must be *defined* before it is called – Bash reads the script top to bottom, so calling a function above its definition fails.

7.4 Function Arguments

Function arguments are accessed using positional parameters, the same special variables also used for script arguments:

Symbol	Meaning
`$0`	Script name
`$1, $2, $3, ...`	Function arguments, by position
`$#`	Number of arguments
`$@`	All arguments, as separate words

Table 7.1: Special Variables in Shell Scripting

Figure 7.1 shows how these map onto an actual function call.

Figure 7.1: How positional parameters map onto the arguments of a function call.

Example:

```
#!/bin/bash
print_args() {
    echo "Number of arguments: $#"
    echo "All arguments: $@"
}
print_args 1 2 3
```

7.5 Returning Values

Functions in Bash **do not** return values the way functions do in C or Python. There are two common ways to get a result out of a function instead:

1. **Using `return` (for an exit status only):** `return` can only report a small integer (0–255), conventionally used to signal success (0) or failure (any non-zero value) – not to hand back a computed value.

```
#!/bin/bash
check_even() {
    if (( $1 % 2 == 0 )); then
        return 0
    else
        return 1
    fi
}

check_even 4
echo "Exit status: $?"   # 0 -> even
```

2. **Using `echo` (to send actual output):** to get a real value – a number, a string, anything – have the function `echo` it, and capture that output with command substitution:

```
#!/bin/bash
square() {
    echo $(($1 * $1))
}

result=$(square 4)
echo "Result: $result"
```

7.6 Variable Scope

Variables inside a function are **global by default** – a function can freely read and overwrite a variable defined outside it. Use the `local` keyword to limit a variable's scope to that function only, so it doesn't leak out or clash with a variable of the same name elsewhere in the script. Figure 7.2 illustrates this with the example below.

Example:

```
#!/bin/bash
my_var="global"

change_var() {
    local my_var="changed inside function"
    echo $my_var
}

change_var
echo $my_var
```

Output:

```
changed inside function
global
```

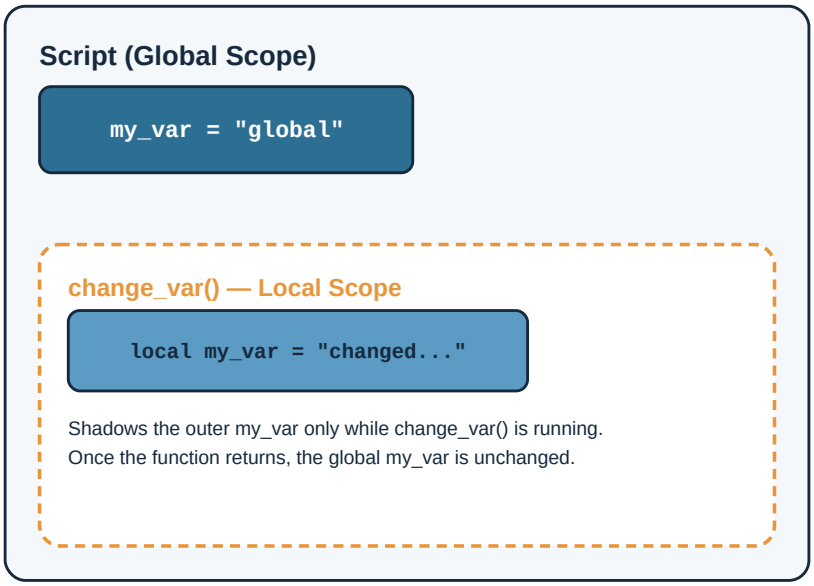


Figure 7.2: A `local` variable only shadows the global one for the duration of the function call.

Note: had `local` been omitted on line 2 of `change_var`, the assignment would have overwritten the global `my_var`, and the final `echo $my_var` would print `"changed inside function"` instead of `"global"`.

7.7 Recursive Functions

Because a Bash function is just a named command, it is allowed to call *itself* – this is called **recursion**. A recursive function needs two things to terminate correctly:

- A **base case**: a condition simple enough to answer directly, without another call.

- A **recursive case**: the function calls itself with an input that moves *closer* to the base case.

Without a base case (or if the input never actually approaches it), the function calls itself forever until Bash runs out of resources.

Example: summing 1 to `n` recursively

```
#!/bin/bash
sum_to() {
    local n=$1
    if (( n <= 1 )); then
        echo 1              # base case
    else
        local rest=$(sum_to $((n - 1)))   # recursive case
        echo $((n + rest))
    fi
}

echo "Sum: $(sum_to 5)"     # Sum: 15
```

Explanation: `sum_to 5` calls `sum_to 4`, which calls `sum_to 3`, and so on down to `sum_to 1` – the base case, which returns `1` directly. Each call then adds its own `n` to the result of the call below it, unwinding back up to the original call.

7.8 Hands-on Practices

Complete the following three practices using the Linux terminal. Record the commands you use, save your scripts, and explain the results where requested. Work inside your `os-lab/` directory.

7.8.1 Practice 1: Defining, Calling, and Passing Arguments to Functions

This practice covers the two ways of defining a function, calling a function, the rule that a function must be defined before it is called, and positional parameters.

1. Create a directory named `chapter7-practice` and move into it. Create a script named `functions.sh` that defines one function named `greet` using the `function` keyword and another function named `greet2` without it. Both should print `Hello, $1!`. Call each of them with a different name, make the script executable, and run it. Then move one of the calls above its function definition, run the script again, observe the error, and fix it.
2. Create a script named `args.sh` that defines a function `print_args` printing the number of arguments (`$#`), all arguments (`$@`), and the first two arguments (`$1` and `$2`). Call it with the arguments `1 2 3`, then with the words `red green blue yellow`, and finally with no arguments at all.
3. Add a function named `list_args` to `args.sh` that uses a `for` loop over `"$@"` to print every argument on its own line in the form `Argument 1: red`, using a counter variable. Call it with at least four arguments.

Now, answer the following questions:

- What are the two ways of declaring a function, and do you need to declare the number or type of its arguments?
- What error did you get when you called a function before its definition, and why does Bash behave this way?
- What did `$#`, `$1`, and `$2` contain when `print_args` was called with no arguments?

7.8.2 Practice 2: Returning Values from Functions

This practice covers returning an exit status with `return`, returning a real value with `echo` and command substitution, and using functions together with conditions and loops.

1. Create a script named `status.sh` that defines a function `check_even`, which returns `0` if its first argument is even and `1` otherwise, using `(( ))`. Call it with `4` and with `7` and print `$?` after each call. Then use the function directly as the condition of an `if` statement to print whether `10` is even or odd. Finally, add a function that executes `return 300`, call it, and print `$?`.
2. Create a script named `values.sh` that defines a function `square`, which prints the square of its first argument, and a function `add`, which prints the sum of its two arguments. Capture their output in variables using command substitution and print the results with messages such as `Result: 16`.
3. Add a function named `max` to `values.sh` that prints the larger of its two arguments. Create an array `numbers` with at least six numbers, then use a `for` loop and the `max` function to find the largest number in the array, and print it.

Now, answer the following questions:

- What did `$?` contain after `return 300`, and what does this tell you about using `return` to send back computed values?
- Why can a function be used directly as the condition of an `if` statement?
- Which way of returning values did you use in `square`, `add`, and `max`, and why?

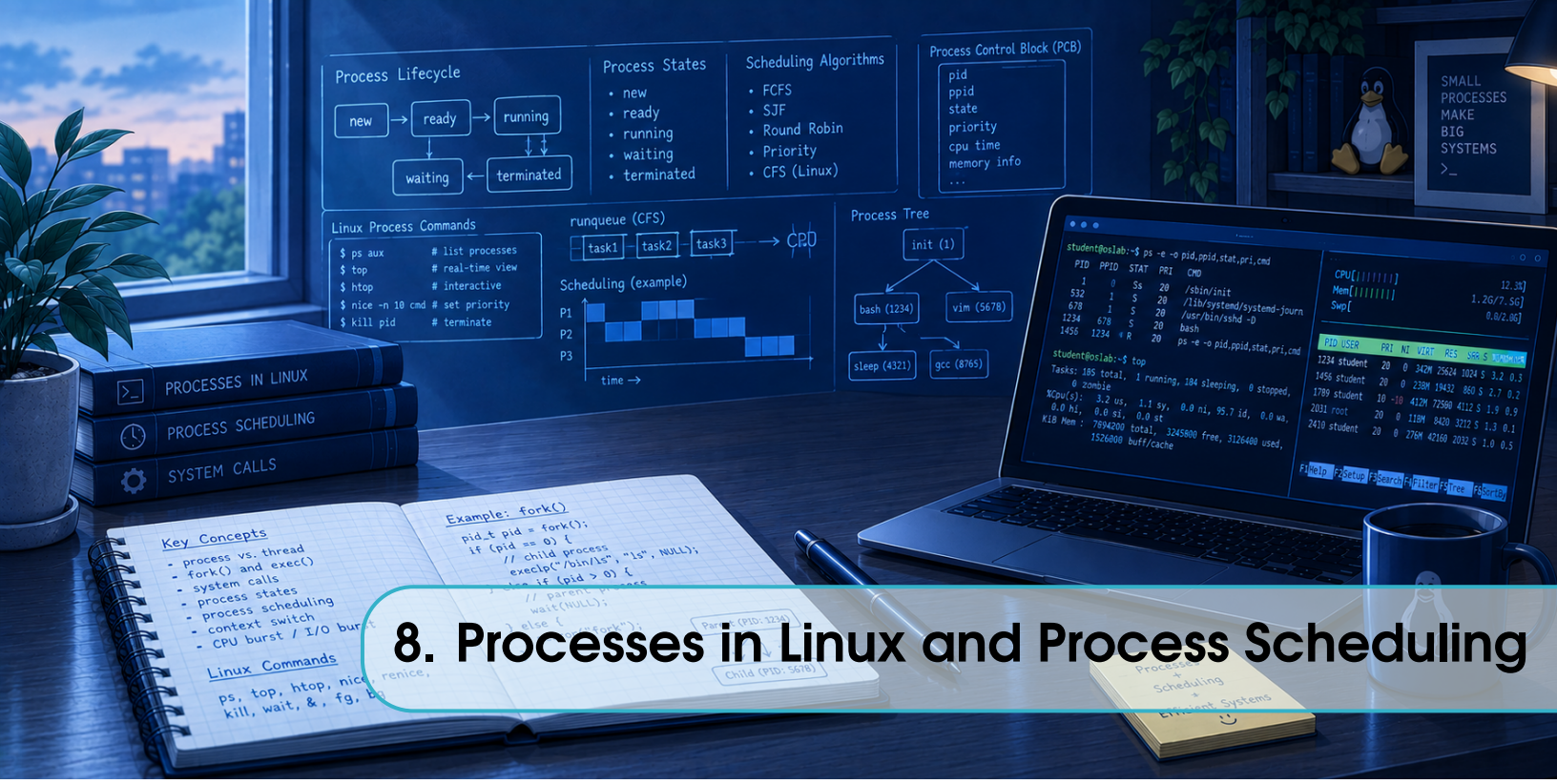
7.8.3 Practice 3: Variable Scope and Recursive Functions

This practice covers global and `local` variables, and recursive functions with a base case and a recursive case.

1. Create a script named `scope.sh` that sets a global variable `my_var="global"` and defines two functions: `change_global`, which assigns a new value to `my_var` without `local`, and `change_local`, which assigns a new value to `my_var` using `local`. Print `my_var` before any call, after calling `change_local`, and after calling `change_global`, and compare the three values.
2. Create a script named `recursion.sh` that defines a recursive function `sum_to` that calculates the sum of the numbers from 1 to `n`, using `local` variables. Print the result for 5 and for 10.
3. Add a recursive function named `factorial` to `recursion.sh`, with a base case for $n \leq 1$ and a recursive case that multiplies `n` by the factorial of `n - 1`. Use a `for` loop over `{1..6}` to print the factorial of every number in the range.

Now, answer the following questions:

- What is the value of `my_var` after each call, and which keyword caused the difference?
- What are the base case and the recursive case of your `factorial` function?
- What would happen if a recursive function had no base case, or if its input never moved closer to the base case?



8. Processes in Linux and Process Scheduling

In multitasking operating systems like Linux, a **process** is the basic unit of program execution – every running program, including every shell script from Chapters 5–7, forms at least one process the moment it runs. The operating system uses mechanisms such as the Process Control Block (PCB), process states, context switching, scheduling, and a handful of system calls to manage these processes.

8.1 What is a Process and What Components Does It Have?

A process is a representation of a running program, with its information stored in a structure called the PCB. The main components of a process include:

- **Program Code:** the executable instructions.
- **Program Counter:** the address of the next instruction to execute.
- **Registers:** the current state of the CPU for this process.
- **Memory Address Space:** including sections such as Text (code), Data (static data), Heap (dynamic data), and Stack (local variables and function calls).

The operating system uses this information to manage processes: pausing or resuming them, and switching between multiple processes running on the same CPU.

8.2 Process Control Block (PCB)

The PCB is a data structure the operating system uses to store information about each process. Its most important components are listed in Table 8.1.

The PCB plays a key role in process management: the operating system uses it to save a process's state when switching away from it, and to restore that exact state later when the process runs again.

PCB Component	Description
PID	Unique process identifier (Process ID).
PPID	Parent process identifier (Parent Process ID).
Process State	States like Running, Ready, or Blocked.
Memory Management Info	Pointers to memory sections (Text, Data, Heap, Stack).
I/O Info	Input/output status, open files, and devices.

Table 8.1: Main Components of a Process Control Block (PCB)

8.3 Process States

A process moves through several distinct states over its lifetime. Figure 8.1 shows the standard transitions between them.

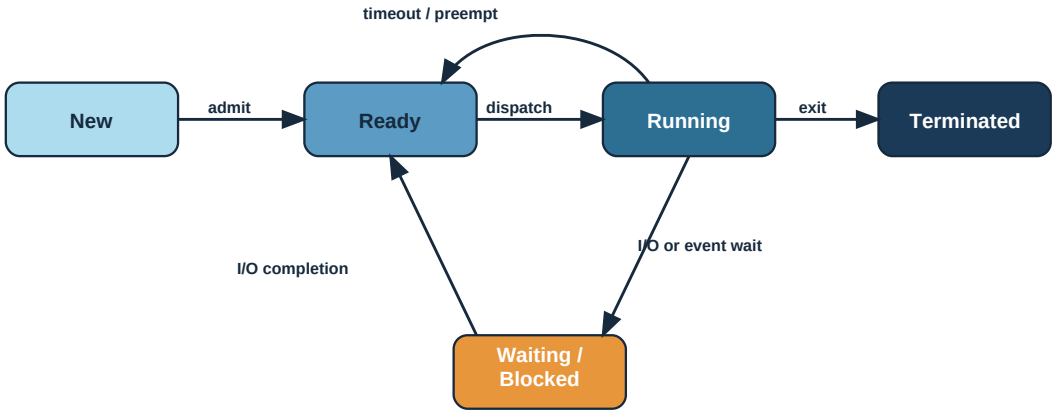


Figure 8.1: Process state transitions: a process moves between these five states as it is created, scheduled, and eventually terminated.

- **New:** the process is being created.
- **Ready:** ready for execution, waiting only for CPU allocation.
- **Running:** currently executing on the CPU.
- **Waiting/Blocked:** waiting for a specific event, such as I/O completion.
- **Terminated:** execution has finished and its resources have been released.

Note that a process can only reach *Running* through *Ready* – it never goes directly from *Waiting* or *New* onto the CPU.

8.4 Process Management System Calls

In this section, you will learn about three essential system calls: `fork()`, the `exec()` family, and `wait()`.

8.4.1 `fork()`

The `fork()` function creates a child process by making a complete copy of the calling (parent) process. Figure 8.2 shows what happens to the two resulting processes.

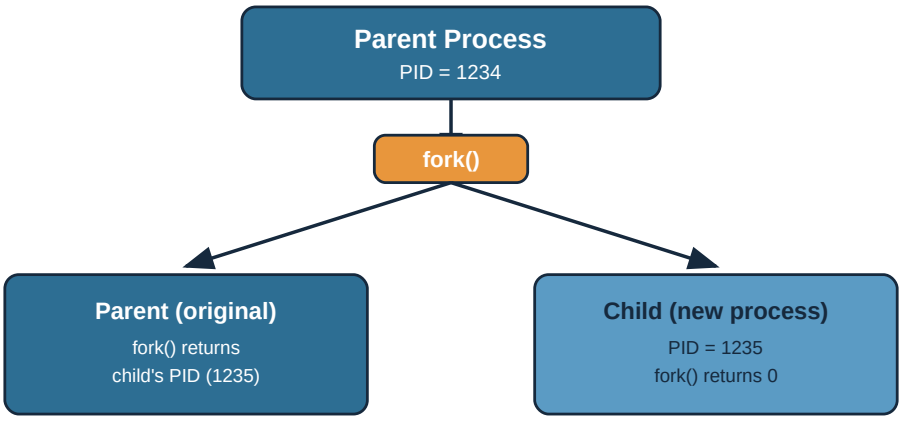


Figure 8.2: A single call to `fork()` returns twice: once in the parent, once in the child – the return value is how each process tells which one it is.

Example Code:

```c
#include <unistd.h>
#include <stdio.h>

int main() {
    pid_t pid = fork();  // Step 1: Call fork() to create a child process.

    if (pid == 0) {      // Step 2: If pid == 0, we are in the child.
        printf("Child Process: PID = %d\n", getpid());
    } else if (pid > 0) {  // Step 3: If pid > 0, we are in the parent.
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    } else {             // Step 4: If fork() fails (pid == -1), handle error.
        perror("fork failed");
    }

    return 0;            // Step 5: End the program for both processes.
}
```

Compilation and Execution:

1. Compile: `gcc fork_example.c -o fork_example`
2. Run: `./fork_example`

Sample Output:

```
Parent Process: PID = 1234, Child PID = 1235
Child Process: PID = 1235
```

Step-by-Step Explanation:

- `fork()` is called, creating the child process; the parent's memory space is copied.
- In the child, `fork()` returns `0`, so the child's branch of the code runs and prints the child message.
- In the parent, `fork()` returns the child's PID, so the parent's branch runs and prints the parent message.
- If an error occurs (e.g. a resource shortage), `fork()` returns a negative value and `perror` displays the error.
- Both processes continue independently from that point on; the order of their output can vary, since it depends on how the scheduler interleaves them.

8.4.2 `exec()` Family

The `exec()` family replaces the calling process's own code and data with a new program. The PID stays the same – only what that PID is running changes.

Example Code:

```c
#include <unistd.h>  // For execvp()
#include <stdio.h>

int main() {
    // Step 1: Define argument array for the new program (here ls -l).
    char *args[] = {"/bin/ls", "-l", NULL};  // NULL is to end the array.

    // Step 2: Call execvp() to replace current process with ls -l.
    execvp(args[0], args);  // If successful, does not return.

    // Step 3: If execvp fails, this code executes.
    perror("execvp failed");
    return 1;
}
```

Sample Output: the output of the `ls -l` command (a detailed file listing).
Step-by-Step Explanation:

- Prepare the arguments: `args[0]` is the program path, and the remaining entries are its parameters.
- `execvp` is called: if it succeeds, the current program's code is replaced immediately, and nothing written after the `execvp` call runs.

- If it fails (e.g. a wrong path), `execvp` returns and `perror` shows the error.

8.4.3 `wait()`

The `wait()` function lets a parent process pause until one of its children finishes, preventing **zombie processes**.

Example Code:

```c
#include <sys/types.h>  // For pid_t
#include <sys/wait.h>   // For wait()
#include <unistd.h>     // For fork() and getpid()
#include <stdio.h>      // For printf()

int main() {
    pid_t pid = fork();  // Step 1: Create child with fork().

    if (pid == 0) {      // Step 2: In child, perform some task
        printf("Child Process is running: PID = %d\n", getpid());
        // You can add time-consuming work here, like sleep(5);
    } else if (pid > 0) {  // Step 3: In parent, wait for child.
        wait(NULL);        // wait() waits until child finishes.
        printf("Child Process finished. Parent continuing.\n");
    } else {
        perror("fork failed");
    }

    return 0;            // Step 4: End the program.
}
```

Sample Output:

```
Child Process is running: PID = 1235
Child Process finished. Parent continuing.
```

Step-by-Step Explanation:

- `fork()` creates the child.
- The child performs its task (e.g. a loop or a `sleep`).
- The parent waits with `wait()`.
- Once the child finishes, the parent continues.

Important Note: if you suspect a script is leaving zombies behind, look for them with:

```
$ ps aux | grep Z
```

8.5 Context Switching

A **context switch** happens whenever the operating system switches the CPU from one process to another.

- Save the current process's state into its PCB.
- The scheduler selects the next process to run.
- Restore that process's state from its own PCB.
- Start (or resume) executing the new process.

Because saving and restoring state takes time that isn't spent running either process, frequent context switching has a real, measurable performance cost – it's a trade-off the scheduler makes to keep the system responsive.

Example Code to Observe the Effect:

```
#include <unistd.h>  // For fork() and sleep()
#include <stdio.h>   // For printf()

int main() {
    fork();          // Step 1: First fork() - creates 2 processes.
    fork();          // Step 2: Second fork() - now 4 processes (tree structure)
    .

    printf("Process PID: %d\n", getpid());  // Step 3: Each process prints its
    PID.
    sleep(2);        // Step 4: sleep to simulate time-consuming work and
    observe switching.

    return 0;        // Step 5: End.
}
```

Sample Output:

```
Process PID: 1234
Process PID: 1235
Process PID: 1236
Process PID: 1237
```

Step-by-Step Explanation:

- First `fork()`: produces the parent and child 1.
- Second `fork()`: each of those forks again, so the parent produces child 2, and child 1 produces child 3.
- All 4 processes now print their message; the order depends entirely on the scheduler.
- Calls to `sleep` free the CPU, giving the scheduler a natural opportunity to switch to another process.
- This program is non-deterministic – the output order can change from one run to the next.

8.6 Displaying Process Structure with `pstree`

The `pstree` command visualizes the hierarchical structure of processes in a tree format.

8.6.1 Why Use pstree?

- It gives a clear, visual picture of process ancestry: which process spawned which children.
- It's useful for debugging multi-process programs, spotting zombie or orphan processes, and understanding overall system resource usage.
- In this lab, it ties directly to the `fork()` examples above: after running one, you can use `pstree` to verify the exact process tree your code created.

8.6.2 Basic Usage

The simplest way to use `pstree` is to run it with no arguments; for more detail, add `-p` to show PIDs.

Sample Command:

```
$ pstree -p
```

- `-p`: displays each process's PID alongside its name.
- This shows the entire system's process tree, starting from the `init` process (PID 1, usually `systemd` on modern Linux).

Chapter 8 introduced the process as the operating system's basic unit of execution.

9.1 Introduction

Multithreading allows a program to execute multiple threads concurrently within the same process.

9.2 Process vs Thread

Figure 9.1 shows the key structural difference: every thread in a process shares that process's code, data, and heap, but each thread still keeps its own stack, registers, and program counter – otherwise, two threads couldn't be running different lines of code at the same time.

Feature	Process	Thread
Memory Space	Completely separate	Fully shared with other threads
System Resources	Separate (files, sockets, etc.)	Shared
Creation & Context Switch Overhead	High	Very low
Independence	Fully independent	Dependent on parent process
Communication Speed	Slow (requires IPC)	Very fast (shared memory)

Table 9.1: Comparison of Processes and Threads

Advantages of Multithreading:

- True parallelism on multi-core CPUs.
- Better responsiveness (e.g. a UI stays active while background work runs on another thread).
- Easy and fast data sharing between threads (no IPC needed, since memory is already shared).

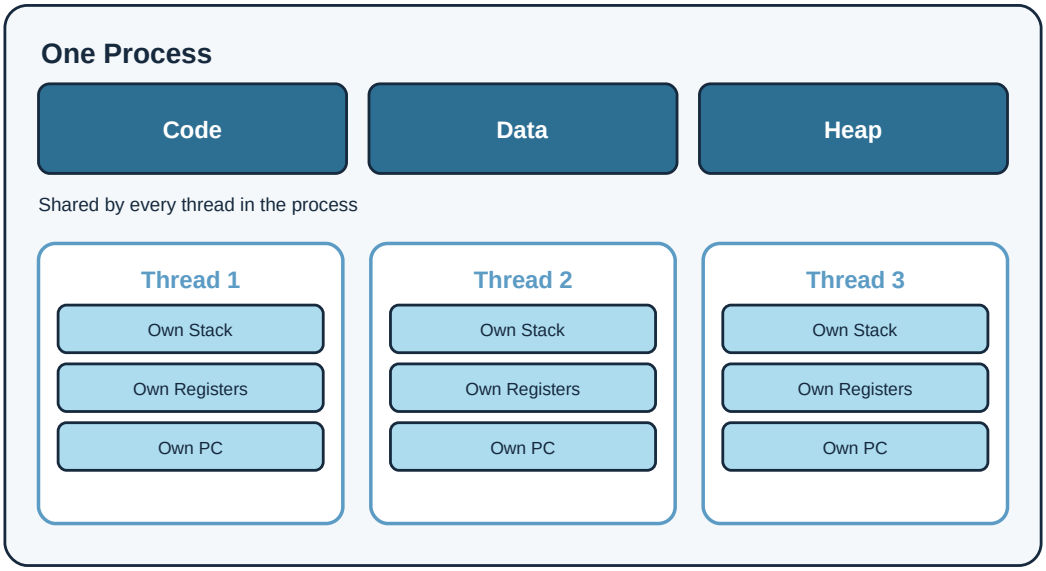


Figure 9.1: A process and its threads: shared code/data/heap, but a private stack, registers, and program counter per thread.

- Much lower memory and creation overhead than spawning separate processes.
- Higher throughput in servers (e.g. handling one client request per thread).

That shared memory is also exactly what makes multithreading dangerous without care – Section 9.4 shows what goes wrong when two threads touch the same data without coordinating.

9.3 POSIX Threads (pthread)

`pthread` is the standard POSIX Threads library for multithreaded programming on Unix/Linux systems.

To use pthread in C:

```
#include <pthread.h>
```

Compile with the pthread flag:

```
$ gcc -pthread program.c -o program
```

Note: the `-pthread` flag is required at compile time even though you only `#included` the header – without it, the linker won't pull in the actual pthread implementation.

Main pthread Functions:

Simple Example: Creating One Thread

```
#include <pthread.h>
#include <stdio.h>
```

Function	Description
`pthread_create()`	Creates a new thread and starts executing the specified function
`pthread_join()`	Waits for the specified thread to terminate
`pthread_exit()`	Terminates the calling thread
`pthread_self()`	Returns the ID of the calling thread
`pthread_detach()`	Detaches a thread (its resources are reclaimed automatically on exit)
`pthread_mutex_init()`	Initializes a mutex
`pthread_mutex_lock()`	Locks a mutex (blocks if it is already locked)
`pthread_mutex_unlock()`	Unlocks a mutex
`pthread_mutex_destroy()`	Destroys a mutex after use

Table 9.2: Common pthread Functions

```
void* my_thread(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t tid;

    pthread_create(&tid, NULL, my_thread, NULL);  // Create thread
    pthread_join(tid, NULL);                      // Wait for thread to finish

    printf("Main thread finished.\n");
    return 0;
}
```

Output:

```
Hello from the new thread!
Main thread finished.
```

Analysis:

- The thread function must return `void*` and accept `void*` – a generic signature that lets you pass and return any kind of data.
- `pthread_create` hands back the new thread's ID through an output pointer, rather than as its return value (which instead reports success or failure).
- `pthread_join` is essential – without it, if `main` exits first, the whole program terminates and the child thread is killed along with it, whether or not it had finished.

Practical Example: Multiple Threads Doing Different Tasks

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void* hello(void* arg) {
    for (int i = 0; i < 5; i++) {
        printf("Hello\n");
        sleep(1);
    }
    return NULL;
}

void* world(void* arg) {
    for (int i = 0; i < 5; i++) {
        printf("      World\n");
        sleep(1);
    }
    return NULL;
}

void* count(void* arg) {
    for (int i = 1; i <= 5; i++) {
        printf("            Counter: %d\n", i);
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;

    pthread_create(&t1, NULL, hello, NULL);
    pthread_create(&t2, NULL, world, NULL);
    pthread_create(&t3, NULL, count, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    printf("All threads finished.\n");
    return 0;
}
```

Sample (interleaved) output:

```
Hello
      World
            Counter: 1
Hello
      World
            Counter: 2
```

```
...
```

This shows real concurrency – the output order is unpredictable, because the threads are genuinely running at the same time (or being rapidly interleaved on a single core), not one after another.

9.4 Race Condition Problem

When multiple threads modify a shared variable without synchronization, their updates can interfere with each other:

```
#include <pthread.h>
#include <stdio.h>

int counter = 0;

void* worker(void* arg) {
    for(int i = 0; i < 1000000; i++)
        counter++;      // <- Dangerous! Race condition
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter: %d\n", counter);   // Usually < 2000000
    return 0;
}
```

What do we expect from this program?

1. We have two threads, each calling the same function (`worker`).
2. Each of them increments the `counter` variable by one, 1,000,000 times.
3. At the end we print the variable and expect to see 2,000,000.
4. But the actual answer is always *less than* 2,000,000.

Why? The statement `counter++` is not atomic – it actually consists of three separate steps:

1. Read `counter`.
2. Increment the value.
3. Write the new value back.

If two threads interleave these steps, they can both read the *same* old value before either writes back – so one thread's increment is silently lost.

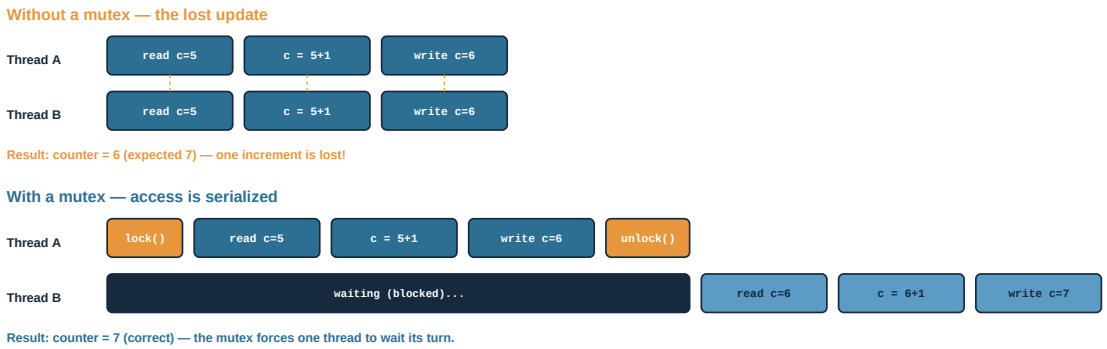


Figure 9.2: The same read-increment-write sequence, without synchronization (top, a lost update) and with a mutex (bottom, correctly serialized).

9.4.1 Solving Race Conditions with a Mutex

To prevent race conditions, we need a synchronization mechanism that ensures only one thread at a time can access a shared resource – the section of code touching that resource is called the **critical section**.

The most frequently used and simplest of these is a **mutex** (short for "mutual exclusion").

A mutex is a lock that protects a critical section of code.

```
#include <pthread.h>
#include <stdio.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* worker_safe(void* arg) {
    for(int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);    // Only one thread can enter
        counter++;
        pthread_mutex_unlock(&lock);  // Release lock
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker_safe, NULL);
    pthread_create(&t2, NULL, worker_safe, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter: %d\n", counter);   // Always exactly 2000000
    return 0;
}
```

Analysis:

- The critical section (the increment) is protected by the mutex.
- Only one thread at a time can execute it, so no update is ever lost.
- Correctness is guaranteed, at the cost of a small performance overhead – every other thread has to wait its turn instead of running freely.

10. Inter-Process Communication (IPC)

10.1 Introduction

By default, modern operating systems isolate processes from one another. Each process has its own independent memory address space, ensuring that a bug or crash in one program cannot easily corrupt the data of another. However, many applications require processes to work together – for instance, the parent/child pairs you built with `fork()` in Chapter 8, or the threads from Chapter 9 (which, unlike processes, already share memory by default and so need IPC far less often).

Inter-Process Communication (IPC) is a set of mechanisms that allows separate processes to exchange data safely and in a structured manner across these isolated boundaries. IPC is essential for cooperating processes, parallel tasks, and any system that requires controlled data sharing.

In the context of execution, processes are either:

- **Independent processes:** they share no data and operate in complete isolation; they do not need IPC.
- **Cooperating processes:** they interact, synchronize, and share data; they require IPC mechanisms to function.

Linux provides several IPC mechanisms, each suited to different types of communication. In this chapter, we will explore Pipes, FIFOs, Message Queues, Shared Memory, Signals, and Sockets. Figure 10.1 previews the key distinction that runs through most of this chapter: whether data passes *through* the kernel, or whether processes reach the same memory *directly*.

10.2 Pipes (Anonymous Pipes)

A pipe is one of the oldest and simplest inter-process communication methods in Unix-like operating systems. Pipes allow a process to transfer a stream of bytes unidirectionally from one point (the writer) to another point (the reader).

Because anonymous pipes only exist in memory and lack a name in the file system, they can **only** be used between related processes (typically a parent and its child). Furthermore, this method is **volatile**: once the processes terminate, the pipe and any unread data are permanently destroyed.

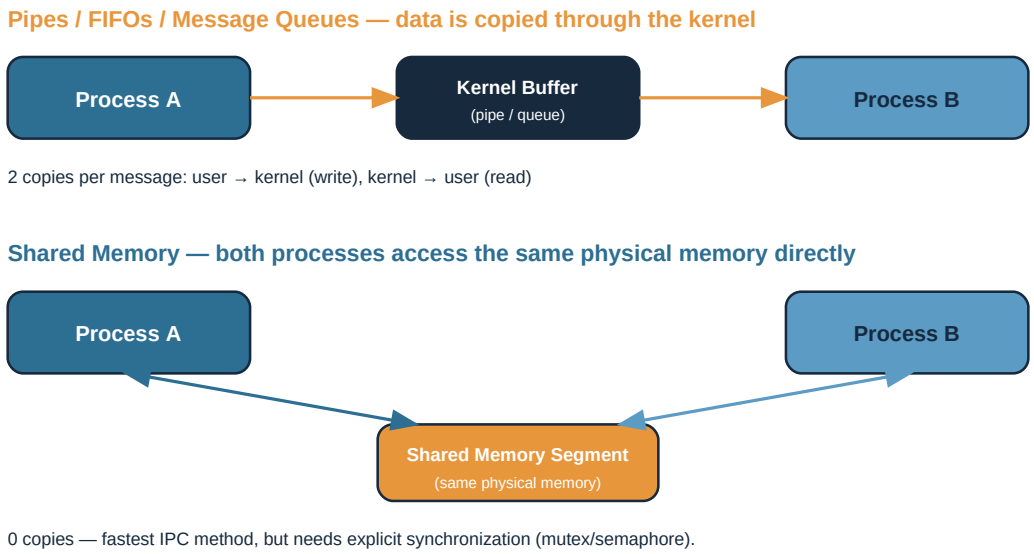


Figure 10.1: Two fundamentally different data paths: kernel-mediated (pipes, FIFOs, message queues) versus direct shared memory.

Function / System Call	Description
`pipe(int fd[2])`	Creates a pipe. `fd[0]` is the read end; `fd[1]` is the write end.
`fork(void)`	Creates a child process that inherits the pipe's file descriptors.
`write(int fd, const void *buf, size_t count)`	Writes `count` bytes from `buf` to the pipe.
`read(int fd, void *buf, size_t count)`	Reads up to `count` bytes from the pipe into `buf`.
`close(int fd)`	Closes a file descriptor. Always close unused ends of a pipe!

Table 10.1: System Calls used for Anonymous Pipes

Figure 10.2 shows how the file descriptors from `pipe()` are used after `fork()`: both processes inherit both ends, but each immediately closes the end it doesn't need.

Example Code: Parent writing to Child

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;
    char buffer[100];
```

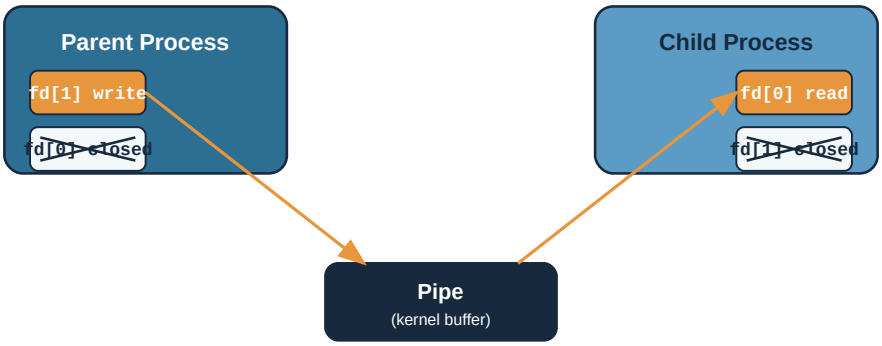


Figure 10.2: After `fork()`, the parent keeps `fd[1]` (write) and the child keeps `fd[0]` (read); the unused end on each side is closed.

```
    // Create the pipe
    if (pipe(fd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    // Fork to create child process
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid > 0) {  // Parent process
        close(fd[0]);  // Close unused read end

        // Write message to the pipe
        char *msg = "Message from parent!";
        write(fd[1], msg, strlen(msg) + 1);  // +1 to include null terminator

        close(fd[1]);  // Close write end

        // Wait for child to finish
        wait(NULL);
        printf("Parent: Message sent successfully.\n");
    }
    else {  // Child process
        close(fd[1]);  // Close unused write end

        // Read message from the pipe
        read(fd[0], buffer, sizeof(buffer));
```

```
        printf("Child received: %s\n", buffer);

        close(fd[0]);  // Close read end
    }

    return 0;
}
```

Step-by-Step Explanation:

- `pipe(fd)` creates the communication channel *before* the fork.
- `fork()` duplicates the process. Now both parent and child have access to `fd[0]` and `fd[1]`.
- Data flows in one direction. To enforce this, the parent closes its read end (`fd[0]`), and the child closes its write end (`fd[1]`).
- The parent uses `write()` to push the string into the pipe, while the child uses `read()` to pull it out.

10.3 FIFOs (Named Pipes)

A FIFO (First-In, First-Out) is a type of pipe that possesses a specific, visible name in the file system. Unlike a normal (anonymous) pipe, which is temporary and restricted to related processes, a FIFO allows completely unrelated processes – even ones started from separate terminals, with no parent/child relationship at all – to communicate, as long as they have the appropriate file permissions.

A FIFO's data still lives in an in-memory kernel buffer, exactly like an anonymous pipe (it is *not* written to disk); only its *name* is visible in the file system. Processes interact with it using standard file operations: `open()`, `read()`, `write()`, and `close()`.

Function / Command	Description
`mkfifo(const char *path, mode_t mode)`	Creates a FIFO special file at `path` (C function).
`mkfifo path`	Same, from the shell – no compilation needed.
`open(const char *path, int flags)`	Opens the FIFO for reading or writing; blocks until the other end is also opened.
`read()` / `write()`	Same semantics as an anonymous pipe, once both ends are open.
`unlink(const char *path)`	Removes the FIFO's name from the file system.

Table 10.2: Functions and Commands used for FIFOs

Example: Two Independent Programs, One FIFO

Unlike a pipe, a FIFO is naturally demonstrated with two *separate* programs run in two separate terminals.

`writer.c`:

```c
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>

int main() {
    mkfifo("myfifo", 0666);  // Create the FIFO (harmless if it already exists)

    int fd = open("myfifo", O_WRONLY);  // Blocks until a reader opens it too
    char *msg = "Hello through a FIFO!";
    write(fd, msg, strlen(msg) + 1);
    close(fd);

    printf("Writer: message sent.\n");
    return 0;
}
```

`reader.c`:

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    mkfifo("myfifo", 0666);  // Create the FIFO (harmless if it already exists)

    int fd = open("myfifo", O_RDONLY);  // Blocks until a writer opens it too
    char buffer[100];
    read(fd, buffer, sizeof(buffer));
    close(fd);

    printf("Reader received: %s\n", buffer);
    return 0;
}
```

Running it:

```
$ gcc reader.c -o reader
$ gcc writer.c -o writer
$ ./reader &        # start the reader first, in the background
$ ./writer           # then run the writer
```

Note: `open()` on a FIFO blocks until *both* a reader and a writer have opened it – this is why running the reader first (with `&`, so it doesn't block your terminal) and the writer second works cleanly.

10.4 System V Message Queues

Message queues allow processes to send and receive structured messages rather than just an unstructured stream of bytes. These queues are stored inside the operating system kernel and act as persistent buffers for messages until they are explicitly deleted.

Unlike pipes, messages in a queue are structured into a **message type** and **message text**. This allows the receiving process to selectively pull messages based on their type, rather than strictly receiving them in the order they were sent (FIFO).

Function	Description
`msgget(key_t key, int msgflg)`	Creates a new message queue or accesses an existing one based on the `key`.
`msgsnd(int msgid, const void *msgp, size_t msgsz, int msgflg)`	Appends a message to the queue.
`msgrcv(int msgid, void *msgp, size_t msgsz, long mtype, int msgflg)`	Receives a message from the queue, filtered by `mtype`.
`msgctl(int msgid, int cmd, struct msqid_ds *buf)`	Performs control operations (e.g., `IPC_RMID` to remove the queue).

Table 10.3: System V Message Queue Functions

Example Code: Typed Message Passing

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>
#include <sys/wait.h>

// Message buffer structure (mtype must be first and of type long)
struct msgbuf {
    long mtype;          // Message type
    char mtext[100];     // Message text/data
};

int main() {
    key_t key;
    int msgid;
    struct msgbuf message;

    // Generate a unique key using ftok (requires a file named "msgfile" to
    exist)
    key = ftok("msgfile", 65);
    if (key == -1) {
        perror("ftok failed");
        exit(1);
    }

    // Create or get the message queue
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget failed");
```

```
        exit(1);
    }

    pid_t pid = fork();

    if (pid > 0) {
        // Parent process - Sender
        message.mtype = 1;
        strcpy(message.mtext, "Hello from System V Message Queue!");

        if (msgsnd(msgid, &message, strlen(message.mtext) + 1, 0) == -1) {
            perror("msgsnd failed");
            exit(1);
        }

        printf("Parent: Message sent successfully.\n");

        // Wait for child to finish
        wait(NULL);

        // Remove the message queue
        msgctl(msgid, IPC_RMID, NULL);
        printf("Message queue removed.\n");
    }
    else if (pid == 0) {
        // Child process - Receiver
        sleep(1);  // Give parent time to send first

        // Note: mtype = 1 means it will only grab messages of type 1
        if (msgrcv(msgid, &message, sizeof(message.mtext), 1, 0) == -1) {
            perror("msgrcv failed");
            exit(1);
        }

        printf("Child received: %s\n", message.mtext);
    }
    else {
        perror("fork failed");
        exit(1);
    }

    return 0;
}
```

Note: For `ftok` to work successfully in this example, a file named `"msgfile"` must exist in the same directory where the program is executed. You can create it using `touch msgfile` before running the code.

10.5 System V Shared Memory

Shared memory is an IPC mechanism that maps a specific segment of physical memory into the address spaces of multiple processes. Because data is written directly to memory rather than copied through the kernel via system calls (like pipes or queues), it is the **fastest** method of data transfer between processes – the bottom panel of Figure 10.1 shows why: there is no intermediate kernel buffer to copy through at all.

Important Warning: Because shared memory allows simultaneous access by multiple processes, it inherently introduces race conditions, in exactly the same way concurrent threads did in Chapter 9. This method requires **synchronization**. Mechanisms such as Mutexes or Semaphores must be used to control access and ensure data consistency. (In the educational example below, a simple `sleep()` is used to delay the child, but this is unsafe for real-world applications).

Function	Description
`ftok(const char *pathname, int proj_id)`	Generates a unique key for System V IPC using an existing file.
`shmget(key_t key, size_t size, int shmflg)`	Creates a new shared memory segment or accesses an existing one.
`shmat(int shmid, const void *shmaddr, int shmflg)`	Attaches the shared memory segment to the calling process's address space.
`shmdt(const void *shmaddr)`	Detaches the shared memory segment from the process.
`shmctl(int shmid, int cmd, struct shmid_ds *buf)`	Performs control operations on shared memory (e.g., `IPC_RMID` to remove).

Table 10.4: System V Shared Memory Functions

Example Code: Writer and Reader via Shared Memory

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <string.h>

#define SHM_SIZE 100

int main() {
    key_t key;
    int shmid;
    char *data;

    // Generate unique key (requires "shmfile.txt" to exist)
    key = ftok("shmfile.txt", 65);
    if (key == -1) {
        perror("ftok failed");
        exit(1);
```

```
    }

    // Create shared memory segment
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    if (shmid == -1) {
        perror("shmget failed");
        exit(1);
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid > 0) {  // Parent process (Writer)
        // Attach to shared memory
        data = (char *) shmat(shmid, NULL, 0);
        if (data == (char *) -1) {
            perror("shmat failed");
            exit(1);
        }

        // Write message to shared memory
        strcpy(data, "Hello from Parent using ftok & shm!");

        printf("Parent wrote: %s\n", data);

        // Detach from process space
        shmdt(data);

        // Wait for child to finish reading
        wait(NULL);

        // Remove shared memory
        shmctl(shmid, IPC_RMID, NULL);
        printf("Shared memory removed.\n");
    }
    else {  // Child process (Reader)
        sleep(1);  // Educational hack to ensure parent writes first

        // Connect using the same key
        key = ftok("shmfile.txt", 65);
        shmid = shmget(key, SHM_SIZE, 0666);

        // Attach to shared memory
        data = (char *) shmat(shmid, NULL, 0);

        // Read and print
        printf("Child read: %s\n", data);
```

```
        // Detach
        shmdt(data);
    }

    return 0;
}
```

10.6 Signals

A signal is a very simple, lightweight, asynchronous message used to control or synchronize the behavior of a process. No complex data payload is transferred via a signal; it is primarily used to tell another process **"do something"** or **"stop."** Despite the lack of data payload, it is still categorized as IPC because it alters process execution and allows minimal communication.

Operating systems use signals internally (e.g., stopping a process for an invalid memory access), and users invoke them regularly via the terminal – every time you press `Ctrl+C` to stop a running program, you are sending it a signal.

Description	Signal	Default Action
Terminal interrupt (Ctrl+C by user)	`SIGINT`	Terminate process
Terminal hangup (e.g., modem disconnect)	`SIGTERM`	Terminate process gracefully
Kill (cannot be caught or ignored)	`SIGKILL`	Terminate process immediately
User-defined signal 1	`SIGUSR1`	Developer-defined behavior

Table 10.5: Common Unix/Linux Signals

Catching a Signal in C

Most signals can be intercepted with a custom *handler* function using `signal()`, instead of letting the default action (usually termination) happen. `SIGKILL` is the one notable exception – it can never be caught, blocked, or ignored, which is exactly why it is used as a last resort.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

// This function runs instead of the default action when SIGUSR1 arrives
void handle_sigusr1(int sig) {
    printf("Caught signal %d (SIGUSR1)! Doing custom cleanup...\n", sig);
}

int main() {
    signal(SIGUSR1, handle_sigusr1);  // Register our handler

    printf("PID = %d. Waiting for a SIGUSR1 signal...\n", getpid());
    while (1) {
        pause();  // Sleep until any signal arrives
    }
```

```
    return 0;
}
```

Sending the signal from another terminal:

```
$ kill -SIGUSR1 <PID>
```

Step-by-Step Explanation:

- `signal(SIGUSR1, handle_sigusr1)` tells the kernel to run `handle_sigusr1` instead of terminating the process when `SIGUSR1` arrives.
- `pause()` puts the process to sleep until *any* signal is delivered – it does no polling, so it costs no CPU while waiting.
- The `kill` command, despite its name, just *sends a signal* – by default `SIGTERM`, but any signal (including a harmless one like `SIGUSR1`) can be specified.
- After the handler function returns, execution resumes right where it left off – back inside `pause()`, which then loops around to `pause()` again.

10.7 Sockets

Sockets are one of the most advanced IPC methods available. While all previous methods restrict communication to processes running on the same physical machine, sockets allow data exchange between two processes regardless of whether they are on the same machine (**Unix Domain Sockets**) or across different machines in a network (**Internet Sockets**).

Unlike standard pipes, sockets natively enable **simultaneous** and **bidirectional** communication, making them the foundational tool for implementing modern server–client, distributed, and networked systems.

Function	Description
`socket(int domain, int type, int protocol)`	Creates an endpoint for communication and returns a file descriptor for it.
`bind()`	Assigns a local address (e.g. an IP and port) to a socket.
`listen()`	Marks a socket as passive, ready to accept incoming connections (server side).
`accept()`	Accepts an incoming connection, returning a new socket for that specific client.
`connect()`	Actively connects to a remote socket (client side).
`send()` / `recv()`	Send and receive data over a connected socket.
`close()`	Closes the socket.

Table 10.6: Core Socket Functions

Note: full socket programming (writing a working client and server) involves enough networking background to deserve its own dedicated lab; this section is here so the vocabulary and function

names above are recognizable when you get there, and so you can see where sockets fit relative to every other IPC mechanism in this chapter.

Wish you all the best, Reza Ahmadizadeh